

REARRANGE: Effort estimation approach for software clustering-based remodularisation

Alvin Jian Jia Tan^{a,*}, Chun Yong Chong^{a,*}, Aldeida Aleti^b

^a School of Information Technology, Monash University Malaysia, Jalan Lagoon Selatan, Bandar Sunway, 47500 Subang Jaya, Selangor, Malaysia

^b Faculty of Information Technology, Monash University, Clayton 3168, VIC, Australia

ARTICLE INFO

Dataset link: <https://figshare.com/s/2ace70cefcb45c28fe11>

Keywords:

Effort estimation
Software remodularisation
Software clustering
Refactoring

ABSTRACT

Context: Most research in software clustering and remodularisation typically concludes by recommending the refactoring operations without further insight into the practicality of the proposed technique. Developers might be hesitant to follow through with the refactoring suggestions due to the uncertainty in the effort needed.

Objective: This work aims to address this gap by introducing an effort Estimation Approach for software clustering-based remodularisation (REARRANGE) to close the loop in extant software clustering and remodularisation research by estimating the time required to carry out the suggested refactoring operations based on the history of the evolution of the software. By providing tangible estimates of refactoring effort in person-hours, we can inform developers of complex and time-consuming refactoring operations that will help prioritise refactoring efforts, allowing practitioners to weave in these activities during sprint planning.

Method: REARRANGE builds a machine learning model to predict effort estimation based on past commit activity which extracts Software Features (lines of code, number of methods), Refactoring Features (refactoring type, source and destination) and Dependency Features (dependencies between classes). REARRANGE is then compared against sanity checks, baseline effort estimation models, and state-of-the-art software estimation models. We also attempt to cross-validate REARRANGE's effort estimation with software developers.

Results: Experimented through 25 open-source Java-based projects, the proposed approach estimated the refactoring effort of the test subjects with a Mean Absolute Error (MAE) of 5.47 person-hours against the MAE of the next-best approach of 453.31 person-hours. Based on a survey conducted among software developers, REARRANGE consistently delivers accurate estimates in 93.6% of cases.

Conclusion: The lack of a direct comparison for REARRANGE highlights the need for a refactoring effort-focused estimation model that provides tangible effort estimates in person-hours for refactoring operations. Only then can developers selectively choose relevant refactoring operations while considering the available time and budget constraints, bridging the gap between software clustering research and real-world application.

1. Introduction

Software clustering is one of the software remodularisation and software architecture recovery techniques that has received substantial attention in recent years [1–8]. There can be many goals for software remodularisation, including, but not limited to, representing the high-level architectural view of the analysed software, removing potential technical debt due to sub-optimal software structure, and reverse documentation of poorly documented systems. Software remodularisation can help software developers and maintainers to understand the interrelationships between software components better.

Most studies that introduce new software clustering algorithms often conclude their studies by presenting the final clustering results, with no follow-up guidance for developers on how or where to begin

the refactoring process and how long the process will take [9–12]. Producing the clustering results recommended by the software clustering algorithm is only part of the process. Only when the software developers implement the suggested refactoring operations the benefits of such software clustering algorithms can be realised. However, developers might be hesitant to follow the suggested refactoring operations due to the uncertain amount of effort needed, especially when the suggestions are “Big Bang” remodularisation (refactoring operations that reorganise most of the system's classes into packages) [13,14].

During the course of a software project, the refactoring actions that often lead to the biggest disruption are typically the ones that developers initially view as simple and fast tasks, only to realise later that they are remarkably complex and require a significant amount of time.

* Corresponding author.

E-mail addresses: alvin.tan@monash.edu (A.J.J. Tan), chong.chunyong@monash.edu (C.Y. Chong), aldeida.aleti@monash.edu (A. Aleti).

<https://doi.org/10.1016/j.infsof.2024.107567>

Received 28 November 2023; Received in revised form 24 August 2024; Accepted 24 August 2024

Available online 30 August 2024

0950-5849/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

The work by Paixao et al. [15] suggests that it is essential to consider the range of acceptable refactoring operations tolerable by developers for the change to occur. Refactoring operations that seem straightforward but escalate in complexity where they span multiple sprints become unmanageable and deter most developers from engaging in future refactoring activities [16,17]. It is important to identify these complex and time-consuming refactoring operations and notify developers beforehand, allowing them to schedule them into the project's timeline adequately.

In a software project's timeline, the most disruptive refactoring operations are usually the ones where developers assume it to be a quick and easy task, but it is exceptionally complex and time-consuming. In this project, it is important to identify these complex and time-consuming refactoring operations and be able to notify developers beforehand, which allows developers to schedule these operations into the project's timeline adequately.

By adding a precise effort estimation of the effort needed in person-hours for each change (refactoring operation), developers and software maintainers would be more confident and comfortable conducting the suggested refactoring operations from the software clustering algorithms, allowing them to choose which of the refactoring operations to implement selectively.

This work aims to introduce an effort Estimation Approach for software clustering-based remodularisation (REARRANGE) to close the loop in extant software clustering and remodularisation research by estimating the effort required for the suggested refactoring operations. REARRANGE aims to build and deploy a machine learning model that can provide developers with refactoring recommendations and effort estimations in terms of person-hours needed to change the current structure of the software to the recommended one.

Software refactoring in a real-world setting involves many other activities, such as integration testing. REARRANGE helps prioritise refactoring efforts by providing a general estimate of the effort required, allowing practitioners to weave these activities into sprint planning. Related user studies [14] show that developers are more inclined towards remodularisation solutions with the inclusion of effort estimation. This leads to our first research question, **RQ1: How can we provide reliable predictions of the effort needed to implement clustering results for refactoring?** The output of RQ1 will be a machine-learning model that estimates the effort needed to perform software refactoring operations.

To ensure the reliability of the proposed effort estimation model, comparative assessments are carried out against sanity checks (Mean and Random guess of the refactoring effort from the collected dataset), baseline effort estimation models (Standardised Accuracy, Variance Relative Error) [18,19], and state-of-the-art software effort estimation models (COCOMOII, Fuzzy Logic). By ensuring that REARRANGE outperforms sanity checks and baseline effort estimation models, it establishes the fact that REARRANGE is capable of delivering a refactoring effort estimate that is significantly more reliable than a general approximation. Without established refactoring effort estimation models, REARRANGE is subjected to a comparative analysis against the most comparable alternative, software effort estimation models. By significantly outperforming software effort estimation models, it highlights the lack and need for a dedicated refactoring effort estimation model.

Finally, to validate the real-world applicability of REARRANGE, the effort estimations derived from REARRANGE are presented to developers overseeing the selected software project (test subjects), wherein they are invited to assess the feasibility of undertaking the recommended refactoring task within the effort level estimated by REARRANGE. This is done by filing issue reports in the respective GitHub repository or Jira. The input provided by the developers overseeing the project enables us to assess the effectiveness of REARRANGE. A public user-study survey to assess the feasibility of undertaking the recommended refactoring task within the effort level estimated by

REARRANGE follows this up on the general applicability of REARRANGE. By implementing these two steps, we can gather feedback from both the developers engaged in the project (test subjects) and external developers regarding the feasibility of utilising REARRANGE in practice. This helps answer the second research question, **RQ2: How can we assess whether developers concur with our effort estimates in a real-world setting?**

The proposed approach has two main goals:

- To help software developers estimate the refactoring effort in terms of person-hours required for the suggested refactoring operations by the clustering methods, and
- To provide software developers with fine-grained predictions on individual refactoring operations that allow them to handpick important and relevant refactoring operations that would benefit the project.

This work aims to provide developers with refactoring recommendations and effort estimations in terms of person-hours needed to change the current structure of the software to the recommended one. Based on our prior work [20], this study introduces notable enhancements and expansions, including an additional section dedicated to the validation by general developers. Our prior work only contains the high-level overview and concepts of REARRANGE. This paper details the methodology and experiment design for estimating refactoring effort. The inclusion of additional validation from general developers as a primary enhancement to our research endeavours is a response to feedback received from our prior work. The survey augmentation aims to delve deeper into the validity and practical utility of REARRANGE, aligning closely with our overarching research objective of facilitating its seamless integration into the daily activities of developers. The incorporation of insights garnered from general developers offers a more comprehensive understanding of REARRANGE's effectiveness and its potential impact on software development practices.

2. Scope

This section outlines the boundaries that frame the basis, interpretation, and application of our research findings. By defining the scope of our study, we aim to provide clarity on the specific contexts and scenarios to which our conclusions are applicable.

2.1. Software clustering algorithms

In this research, the focus is on comparing and evaluating the effectiveness of different variants of agglomerative hierarchical clustering algorithms and search-based clustering algorithms because (i.) agglomerative clustering and Bunch (which is a search-based software clustering algorithm) are two of the most popular clustering algorithms, as discussed in the work by Teymourian [21], and (ii.) the results produced by different software clustering algorithms exhibit significantly different structure and behaviour, which are difficult to compare directly. Comparable algorithms share common clustering objectives, yielding comparable results that can be assessed against a standard authoritative decomposition. On the other hand, non-comparable algorithms pursue distinct clustering objectives, leading to substantially divergent outcomes that cannot be directly benchmarked against an authoritative decomposition [22]. In this research, we undertake additional efforts to ensure a fair comparison between hierarchical agglomeration clustering and Bunch, employing MoJoFM as an evaluation metric. This research omits the inclusion of ACDC and LIMBO in the comparative analysis due to their dissimilar clustering objectives, rendering it unfair and unfeasible to subject them to evaluation through MoJoFM. ACDC endeavours to allocate each unclustered software entity to the subsystem with the highest dependency, while LIMBO strives to form clusters that maximise information encapsulation about the features within. The nature of the outputs generated by these software clustering algorithms prevents a fair evaluation using MoJoFM.

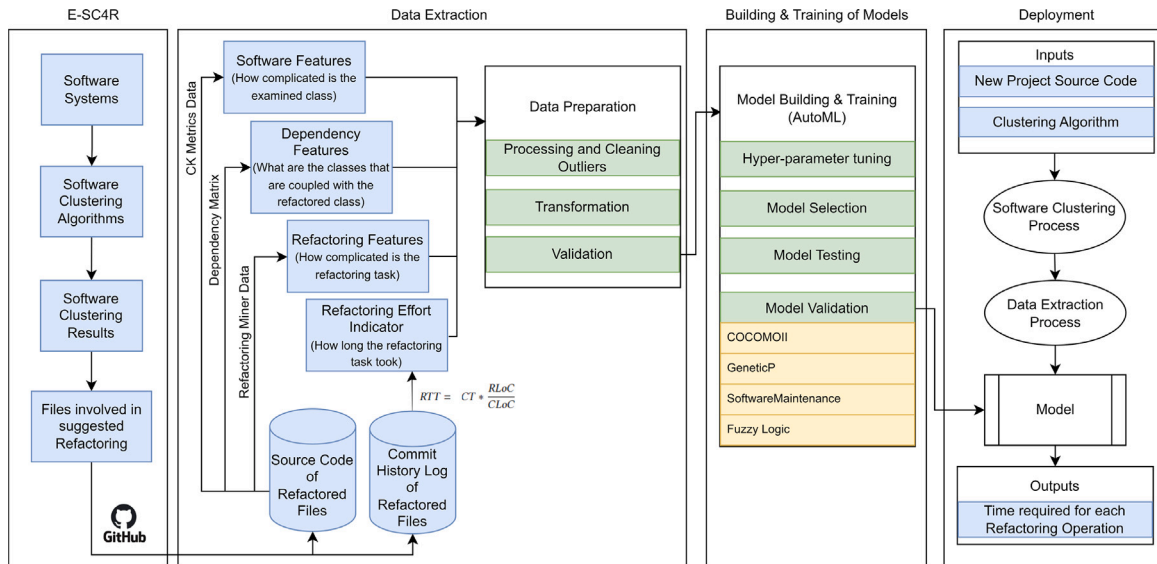


Fig. 1. Overall workflow of REARRANGE.

2.2. Programming languages

This research primarily focuses on software clustering and effort estimation for Java projects. This is due to the reliance on underlying tools and packages for data collection tailored primarily for Java projects, thus excluding consideration for Python projects, another widely adopted programming language. Nevertheless, our proposed methodology can be adapted to Python projects with the appropriate tools and packages to gather the necessary data.

2.3. Effort estimation

Regarding effort estimation, REARRANGE addresses only the implementation-related effort for code changes. However, a comprehensive refactoring process involves various additional tasks, such as code reviews, backward compatibility testing, and integration testing. With improved data collection mechanisms for these activities, it becomes feasible to augment REARRANGE with supplementary models and modules to encompass the entire effort estimation spectrum.

3. The REARRANGE approach

REARRANGE improves the effectiveness of software clustering and remodularisation techniques by bridging the gap between software remodularisation research and real-world application. Estimated efforts provide developers with more confidence in implementing the suggested changes.

The purpose of the proposed effort estimation approach is to **predict the effort required in person-hours to carry out the suggested refactoring operations produced by the software clustering approaches**. The proposed approach performs effort estimation based on the past refactoring operations performed by the developers, which involves four main steps, depicted in Fig. 1.

- 1. Software Clustering:** Software clustering algorithms are applied to the software systems chosen for this study for refactoring purposes. The clustering results identify the source codes involved in the suggested refactoring.
- 2. Data Extraction:** From the source codes and the commit history logs of the refactored files, we collect *Software Features*, *Refactoring Features*, and *Dependency Features* that are important in determining the complexity and size of the refactoring operations, which in turn affects the estimated refactoring effort.

These will be discussed in detail in Sections 3.5.1, 3.5.3, and 3.5.2 respectively.

- 3. Building and Training of Models:** An effort estimation model is then trained based on the features extracted in the previous step, predicting the efforts required for a refactoring operation.
- 4. Application:** In a real-world setting, developers would input the source codes for their personal software system. They would receive refactoring suggestions and the effort required for each suggested operation.

The boxes in Fig. 1 represent the artefacts, while the arrows are the steps for creating the relevant artefacts. The following subsections describe our proposed approach for each artefact and steps in detail.

3.1. Software systems

The software systems used in this study are chosen pseudo-randomly from the pool of the most active GitHub projects to provide a meaningful representation of different types of open-source projects with various sizes and complexities. The selection process is detailed in Section 4.2.

3.2. Software clustering algorithms

Different clustering algorithms proposed in the existing literature address the software remodularisation problem differently depending on the individual interpretation of a meaningful and effective software clustering result. These algorithms usually aim to achieve common goals that focus on improving software modularity, quality, and software comprehension [10,23,24]. In existing studies, it has been shown that depending on the clustering mechanism, different software clustering algorithms tend to produce significantly different clustering results, but at the same time, they might also offer equally valid high-level abstraction views of the analysed software [9,11,22,25].

This work focuses on estimating the effort required to follow the suggested clustering results from agglomerative hierarchical clustering algorithms and Bunch algorithms because they are the most widely used generic clustering algorithms and search-based algorithms [9,23,26–28]. In our proposed approach, we only take into account the most suitable clustering algorithm during the effort estimation process. As such, before adopting REARRANGE to estimate the refactoring effort needed to follow through with the clustering result, we must identify a way to shortlist the most optimal clustering algorithm.

The E-SC4R [29] framework discussed in the previous section is used to determine the most optimum clustering algorithm via an objective assessment among the chosen software clustering techniques. E-SC4R provides information on the suitability of clustering algorithms based on software features, enabling the selection of the most suitable algorithm and configuration. Each algorithm's clustering results and configuration are compared with the ground truth. The comparison is measured based on the MoJo [30] family of metrics, which measures the number of Move and Join operations needed to convert the clustering results to the proxy ground truth. E-SC4R follows the following steps to determine the most suitable clustering algorithm for a software system:

1. Ground truth generation to establish an ideal package structure.
2. Exhaustive performance testing to determine the most suitable clustering algorithm and hyperparameters.
3. Extraction of software features (e.g. lines of code).
4. A genetic algorithm determines the most important software features influencing the suitability of a clustering algorithm. The best subset of features discriminates between high-performing and low-performing clustering algorithms.
5. Relationship between algorithms and hyperparameters are mapped to the most important software features using a Support Vector Machine (SVM).
6. SVM would determine the most suitable clustering algorithm for new unseen software systems via identified important software features.

The E-SC4R framework is tested on 30 open-source systems and is shown to outperform software clustering algorithms on a default setting with a MoJo [29,30] of 73.23 compared to 195.67 by default. This provides a more efficient process as compared to a trial-and-error approach. Using E-SC4R, we can automatically identify the most optimum clustering algorithm and associated parameters, such as linkage algorithm, distance metric, or fitness function, from a pool of clustering algorithms. The following example is an output from E-SC4R using the open-source project DBeaver, a lightweight SQL client, as an input.

- Agglomerative Hierarchical Configuration
 - Linkage Algorithm = Single Linkage
 - Distance Metric = Cosine Metric
 - Cluster Divisor = 25
- Bunch Configuration
 - Search Algorithm = HillClimbing
 - Fitness Calculator = TurboMQIncrW

In the example shown above, we illustrate a scenario such that developers are planning to choose between Agglomerative Hierarchical and Bunch Clustering algorithms to perform modularisation on the project DBeaver. Using E-SC4R, the most optimum clustering result can be achieved using the abovementioned parameters, which gives the most stable package structure defined in E-SC4R.

We use the aforementioned software clustering algorithm and configuration to produce the most optimum clustering results. The suggested refactoring operations will be passed into the next step of our proposed approach.

3.3. Software clustering results

From the software clustering results, we convert the current package structure to the recommended structure suggested by the most optimum clustering algorithm.

To interpret the output of the clustering algorithms, we first convert the project structure into the following:

- contain code `NotNull.java`

- contain `utils ArgumentTokenizer.java`
- contain `io BOMInputStream.java`

where it can be interpreted as: the folder “code” contains `NotNull.java`, and the folder “utils” contains `ArgumentTokenizer.java`.

Depending on the clustering algorithms used, the sample outputs of each algorithm are given below.

Agglomerative Hierarchical Algorithm:

- contain code `NotNull.java`
- contain `utils ArgumentTokenizer.java`
- contain `utils BOMInputStream.java`

This is interpreted as `BOMInputStream.java` should be moved to the package “utils” (highlighted in blue). `NotNull` and `ArgumentTokenizer` has no change given the same parent folder from the original project structure and the algorithm's output.

Bunch Algorithm:

- contain code `NotNull.java`
- contain `utils ArgumentTokenizer.java`
- contain code `BOMInputStream.java`

This is interpreted as `BOMInputStream.java` should be moved to the package “code”. Similarly, given the same parent folder, there is no change to `NotNull` and `ArgumentTokenizer`.

REARRANGE's goal is to determine each individual effort required to refactor the current software's package structure to the preferable package structure suggested by the clustering algorithm. This is achieved by estimating the effort for each move of the individual class packages.

3.4. Files involved in suggested refactoring

The files of the suggested refactoring will be filtered. Building upon the previous example, “`BOMInputStream.java`” will be specifically isolated for data extraction during the subsequent stage.

3.5. Data extraction

The time taken for a software refactoring operation is generally highly dependent on the size and complexity of the operation [13,31]. Therefore, to develop a reliable and accurate effort prediction model, we must look at different aspects of the refactoring operations, including the target class's complexity and the refactoring operation's nature.

To better model the relationship between the source files and provide an accurate effort estimation, we propose to **extract software features, refactoring features, and dependency features to form the basis of the effort estimation model**. Software features such as lines of code, the number of public methods, and the number of static methods are extracted to estimate the complexity of the classes involved in the refactoring operation. Refactoring features are extracted to represent the size and complexity of the refactoring operation itself. Dependency features measure the coupling of the target class to be refactored with other classes in the project. The hypothesis is that if a refactoring operation involves classes with entangled dependencies, the complexity of the related classes will also need to be considered when estimating the overall refactoring effort. The following subsections explain in detail how each feature is extracted.

3.5.1. Software features

For each refactoring operation in the analysed project, the software features of the classes involved in the refactoring operation are

Table 1
Description of software features.

Object oriented features	
WMC	McCabe's complexity. It counts the number of branch instructions in a class.
DIT	Depth Inheritance Tree, counts the number of parent a class has.
Number of Fields	Counts the number of fields. Specific numbers for total number of fields, static, public, private, protected, default, final, and synchronised fields.
Number of Methods	Counts the number of methods. Specific numbers for total number of methods, static, public, abstract, private, protected, default, final, and synchronised methods. Constructor methods also count here.
Number of visible methods	Counts the number of visible methods. A method is visible if it is not private.
NOSI	Number of static invocations, counts the number of invocations to static methods.
RFC	Response for a class, counts the number of unique method invocations in a class.
LOC	Lines of code. It counts the lines of count, ignoring empty lines and comments (i.e., its Source Lines of Code, or SLOC).
TCC	Tight Class Cohesion, measures the cohesion of a class with a value range from 0 to 1. TCC measures the cohesion of a class via direct connections between visible methods, two methods or their invocation trees access the same class variable.
LCC	Loose Class Cohesion, similar to TCC but it further includes the number of indirect connections between visible classes for the cohesion calculation. Thus, the constraint $LCC \geq TCC$ holds always.
No. of Returns	The number of return instructions.
No. of Loops	The number of loops (i.e., for, while, do while, enhanced for).
No. of Comparisons	The number of comparisons (i.e., == and !=).
No. of try or catches	The number of try or catches.
No. of () expressions	The number of expressions inside parenthesis.
String literals	The number of string literals (e.g., "John Doe"). Repeated strings count as many times as they appear.
Quantity of Number	The number of numbers (i.e., int, long, double, float) literals.
Quantity of Math Operations	The number of math operations (times, divide, remainder, plus, minus, left shift, right shift).
Quantity of Variables	Number of declared variables.
Max nested blocks	The highest number of blocks nested together.
Number of unique words	Number of unique words in the source code. Counts the number of words in a method or class, after removing Java keywords. Names are split based on camel case and underline.
Number of Log Statements	Number of log statements in the source code. The counting uses REGEX compatible with SLF4J and Log4J API calls.
Has Javadoc	Boolean indicating whether a method has javadoc.
Usage of each variable	How often each variable was used inside each method.
Usage of each field	How often each local field was used inside each method, local field are fields within a class (subclasses are not included). Also indirect local field usages are detected, indirect local field usages include all usages of fields within the local invocation tree of a class e.g. A invokes B and B uses field a, then a is indirectly used by A.
Method invocations	All directly invoked methods, variations are local invocations and indirect local invocations.

extracted using the Java CK (Chidamber and Kemerer) code metrics calculator [32,33]. The tool calculates simple size metrics such as the number of methods, lines of code, and the number of private fields, to more complex measures such as depth of inheritance, coupling between objects, and other CK suites of metrics. Table 1 shows a subset of the features used in this section and their definitions for illustration purposes. The complete list of metrics, along with its description, can be found on the tool's GitHub page.¹

3.5.2. Dependency features

We employ *Depends* [34] to extract dependencies between classes in the analysed software to improve the reproducibility of our research findings. *Depends* gathers syntactical relations among source code entities such as files and methods² and extracts the following types of dependencies from a pair of classes: Import, Contain, Parameter, Call, Return, Throw, Implement, Extend, Create, Use, Cast, ImplLink, Annotation, Mixin.

We then generate a $N \times N$ (N = number of classes) matrix that denotes the relationships between all classes. Sample data extracted from *Depends* is shown in Table 2, which is then aggregated into the relationships between the clustering entities shown in Table 3.

We do not attempt to distinguish between the different dependencies identified by *Depends*. Instead, we aggregate all dependencies to

Table 2
Examples of relationships extracted from Depends.

src	dest	Cast	Call	Return	Use	Contain	Import	Extend	Implement
a.java	b.java	1	8	4	9	0	0	6	3
b.java	c.java	1	0	0	8	1	0	1	5

Table 3
Example of a relationship matrix between clustering entities aggregated from Table 2.

	a.java	b.java	c.java
a.java	0	31	11
b.java	31	0	16
c.java	11	16	0

represent the dependency strength between two classes. By constructing the dependency matrix, we can identify the dependencies between the target class (class to be refactored) and the neighbouring associated classes.

Using a.java and c.java as an example in Table 3, when both files are involved in the refactoring process, the total dependencies for each file are given below:

- a.java = 42 (31 + 11)
- c.java = 16 (9 + 4 + 3)

¹ <https://github.com/mauricioaniche/ck>

² <https://github.com/multilang-depends/depends>

The effort for refactoring `a.java` would be more substantial than the effort needed for `c.java` due to the larger number of dependencies with other classes. By refactoring `a.java`, one would need to make sure that the changes do not affect the other 42 classes that interact with `a.java`, which poses a harder challenge compared to `c.java` which only interacts with 16 other classes. The number of underlying dependencies with other classes adds to the complexity of the refactoring operation, as we need to account for the different relationships of the refactored class with its associated classes during the refactoring operation.

3.5.3. Refactoring features

Refactoring is widely defined as code changes specifically focusing on the code structure, with the aim of enhancing internal code quality or facilitating future extensions. We use version 3.0.4 of Refactoring Miner [35–37] to detect refactoring operations applied throughout the evolution history of a project. Refactoring Miner records all refactoring operations carried out between each minor version of the project, from the first to the last release.³ RefactoringMiner is able to detect refactoring operations up to 95% accuracy. Refactoring Miner extracts the following information:

1. Refactoring type,
2. Original location of the affected class,
3. New location post refactoring,
4. Number of lines involved in the refactoring operation.

A sample of the data extracted from Refactoring Miner is detailed in Section 4.3.

3.6. Estimating the refactoring effort

In this section, a proxy effort is generated in estimating the refactoring effort. Various ways exist to assess the effort required for a refactoring operation [38–40]. Typically, a machine learning model learns from a reference model, which refers to a known refactoring effort required as a baseline for comparison.

One way of accessing the reference model is through feedback from domain experts [38], where they can estimate the time needed to perform the refactoring operations based on their prior knowledge in the analysed system. However, this approach is difficult to realise because refactoring operations are usually coupled with other tasks such as bug fixing and feature development. It is difficult for domain experts to isolate the refactoring effort from the overall commit effort. Obtaining feedback from domain experts is also a very time-consuming task, where the scope of the research would have to be narrowed down to several selected case studies, which will impact the generalisation of the proposed approach.

When inputs and feedback from domain experts to create a proxy measure for refactoring effort is not available, several studies have chosen to use a percentage of the commit time [39,40]. These studies have introduced proxy methods to obtain a close approximation for bug-fixing effort estimation, and have successfully shown the feasibility of such an approach in their experiment results. By leveraging insights from the work by Martini et al. and Sellitto et al. we introduce Eq. (1) as an approximation for refactoring effort estimation. Drawing parallels from their approach, a developer is expected to contribute eight working hours per day, corresponding to 8 person-hours for our effort estimation purposes. **If 25% of the committed lines of code are for refactoring activities, it corresponds to 2 person-hours of refactoring effort out of the total 8 person-hours.** We take the percentage of lines of code dedicated for refactoring activities over the total lines committed as a proxy percentage for refactoring effort from the total development effort.

The percentage of the commit time is equivalent to the percentage of the lines of code involved in the refactoring operation over the total number of lines of code committed given in Eq. (1), which is adopted from the work by Martini et al. [39].

$$RTT = CT \times \frac{RLoC}{CLoC} \quad (1)$$

RTT is defined as *Refactoring Time Taken*, CT represents the *Commit Time*, RLoC represents the *Refactored Lines of Code*, and CLoC is the *Commit Lines of Code*. This method is less expensive than retrieving the ground truth from developers in a controlled environment, i.e., recruiting developers to perform specific refactoring tasks and capturing the time taken for each task to be used as the ground truth. To illustrate how the refactoring effort indicator is calculated, we use two example projects below.

The Commit Time is given by the duration between the commit that contains the refactoring of the developer or committer and the previous commit of the developer or committer. The precise amount of time that developers dedicate to refactoring operations remains uncertain due to several factors. Firstly, open-source refactoring endeavours exhibit substantial variability, contingent upon the level of commitment involved. Secondly, data on this is generally accessible primarily for proprietary projects, limiting comprehensive insights. Furthermore, refactoring operations necessitate engagement in numerous downstream activities, including testing and verification, further obscuring the quantification of developer time dedicated to such efforts [41]. This proxy is most effective for full-time developers working on the project. For example, the developer *Scordio* working full time on *Assertj-core* in 2020, with daily commits.⁴

3.6.1. An example of a small refactoring commit

Taking Apache Camel project and one of its commits⁵ as an example with commit time being the duration between the commit that contains the refactoring and the previous commit. We substitute the following into Eq. (1).

- Commit Time (CT) = 9 person-hours,
- Refactored Lines of Code (RLoC) = 25 lines,
- Commit Lines of Code (CLoC) = 102 lines.

The Refactoring Time Taken (RTT) for this commit is given as $9 \times \frac{25}{102} = 2.205$ person-hours.

The refactoring operations that have been carried out are listed within Table 4. The 12 refactoring operations are renaming operations that developers can complete quickly. As such, the ground truth of 2.205 person-hours calculated using Eq. (1) provides a reasonable estimate of the time needed to perform the refactoring operation.

3.6.2. An example of a large refactoring commit

Taking Apache Cassandra and one of its commits⁶ as another example with commit time being the duration between the commit that contains the refactoring and the previous commit. We substitute the following into Eq. (1)

- Commit Time (CT) = 146 person-hours,
- Refactored Lines of Code (RLoC) = 237 lines,
- Commit Lines of Code (CLoC) = 482 lines.

The Refactoring Time Taken (RTT) for this commit is given as $146 \times \frac{237}{482} = 71.78$ person-hours. The refactoring operations that have been carried out are listed below.

⁴ <https://github.com/assertj/assertj/commits?after=d3b7bfad647c54b27e9bdeecb57500cd2896581e+334&author=scordio>
Scordio's Daily Commit Activity in August 2020.

⁵ [c089d566839f168a273847df3c2c98c9b8f7a987](https://github.com/assertj/assertj/commit/c089d566839f168a273847df3c2c98c9b8f7a987)

⁶ [6cd2d07d9ae00fafa460fa1613264c43a283e24d](https://github.com/apache/cassandra/commit/6cd2d07d9ae00fafa460fa1613264c43a283e24d)

³ <https://figshare.com/s/0d24fa70b21104e7dcf1>

Table 4
Refactoring operations carried out for Apache Camel commit c089d5.

Refactoring type	Refactoring details
Rename Class	AbstractRabbitMQIntTest.java to RabbitMQITSupport.java
Rename Class	RabbitMQBasicIntTest.java to RabbitMQBasicIT.java
Rename Class	RabbitMQConsumerIntTest.java to RabbitMQConsumerIT.java
Rename Class	RabbitMQConsumerPooledExchangeIntTest.java to RabbitMQConsumerPooledExchangeIT.java
Rename Class	RabbitMQConsumerQueuesIntTest.java to RabbitMQConsumerQueuesIT.java
Rename Class	RabbitMQConsumerRoutingKeyIntTest.java to RabbitMQConsumerRoutingKeyIT.java
Rename Class	RabbitMQConsumerTopicIntTest.java to RabbitMQConsumerTopicIT.java
Rename Class	RabbitMQInOutIntTest.java to RabbitMQInOutIT.java
Rename Class	RabbitMQPollingConsumerTest.java to RabbitMQPollingConsumerIT.java
Rename Class	RabbitMQProducerIntTest.java to RabbitMQProducerIT.java
Rename Class	RabbitMQProducerNullBodyIntTest.java to RabbitMQProducerNullBodyIT.java
Rename Class	RabbitMQProducerToDIntTest.java to RabbitMQProducerToDIT.java

Table 5
Refactoring operations carried out for Apache Cassandra commit 6cd2d0.

Refactoring type	Refactoring details
Extract Class	cql3/validation/operations/SelectTest.java to cql3/DistinctQueryPagingTest.java
Move Method	public testSelectDistinct() from SelectTest.java to DistinctQueryPagingTest.java
Move Method	public testSelectDistinctWithWhereClause() from SelectTest.java to DistinctQueryPagingTest.java
Move Method	public testSelectDistinctWithWhereClauseOnStaticColumn() from SelectTest.java to DistinctQueryPagingTest.java
Extract Variable	SinglePartitionPager from public state() from MultiPartitionPager.java
Extract Variable	readCommand from public fetchPageInternal() from AbstractQueryPager.java
Extract Variable	readCommand from public fetchPage() from AbstractQueryPager.java
Extract Variable	readCommand from public fetchPageUnfiltered() from AbstractQueryPager.java
Rename Variable	PagingState to pagerState from MultiPartitionPager.java

The refactoring operations included in Table 5 are relatively more complicated than in Table 4. They involve the moving of classes, methods, and the extraction of variables from different methods, which is reflected in the substantially larger *RLoC*. These classes and methods to be refactored have varying dependencies with other classes and methods inside and outside their respective packages. Developers would need to implement these changes without changing the underlying functionality of the software system. Compared to the previous example, the overall increase in complexity justifies the longer time needed for refactoring of 71.78 person-hours. This shows that the ratio between *RLoC* and *CLoC* increases proportionally with refactoring complexity, which allows the ratio to be taken as a proxy to measure refactoring effort.

Due to its proxy nature, offering precise justifications regarding variance and confidence intervals may pose a challenge. Nonetheless, Eq. (1) is one of the more reliable representative effort estimation for the “actual refactoring” effort based on existing literature. The increase in refactoring complexity when *RLoC* increases over *CLoC* shows that the percentage of *RLoC* over *CLoC* can be taken as a proxy to the percentage of refactoring effort over the overall commit effort, which provides us with a proxy measure for Refactoring Effort.

3.7. Data preparation

The extracted data from Section 3.5 are then processed and cleaned. The collected data are organised and transformed into a structured and useable format for the next step. The data goes through the following steps:

- 1. Processing and Cleaning Outliers:** Once the data have been extracted, they must be processed and cleaned. This involves removing irrelevant or duplicate entries, handling and imputing missing values, correcting inconsistencies or errors in the data, and standardising formats. The aim is to ensure the data is accurate, complete, and ready for further analysis. Comprehensive details are presented in Section 4.4.
- 2. Transformation:** The next step is to transform the processed data into a format suitable for modelling. This transformation involves applying various techniques to manipulate the data and make them more amenable for analysis. The specifics of these techniques are detailed in Section 4.5.

3.8. Effort estimation model training

Finally, state-of-the-art machine learning models are trained to estimate the refactoring effort based on the features extracted in Section 3.5, which includes software features, refactoring features, and dependency features. In general, machine learning models can be categorised into classification and regression models. For REARRANGE, classification-based models provide developers with an output of two (or more) classes, classifying each refactoring effort into a minor or major one. Regression-based models give developers a predicted value of the person-hour required for the refactoring effort.

In REARRANGE’s approach, we have decided to work with regression models as developers could determine the number of person-hours required to implement the specific refactoring operation. Using sprints from Agile-based software development as an example, developers would find it easier to work with the effort estimation outputs from regression-based models. Developers can attend to and select refactoring tasks based on the duration of their sprint. For example, for a 1-week sprint, they can commit to refactoring tasks that add up to 40 person-hours during sprint planning. Classification-based models may not be appropriate in this scenario, as developers would only be able to determine whether it is a major or minor refactoring operation from the output of the classification model without a concrete time frame. Categorising the refactoring effort into a minor or major refactoring does not help the developers understand the tangible efforts required to incorporate the clustering results or refactoring operations in their sprint planning. Developers might still hesitate to incorporate major refactoring tasks, where there is no way to know whether the major refactoring task can be completed within 40 person-hours.

To assist us with the training of the effort estimation regression model, we leverage H2O AutoML [42] to learn the relationship between features from Section 3.5 and the refactoring effort required from Section 3.6. H2O AutoML trains multiple models simultaneously, including GBMs, Random Forests, Deep Neural Networks, and GLMs using a combination of fast random search and stacked ensembles with results comparable to Bayesian optimisation or genetic algorithms [42]. Using AutoML, we can automatically choose the best-performing learning algorithm for refactoring effort prediction. By gathering the predictors and the effort required in person-hours for each refactoring operation as a data point, we can train the machine learning model to predict

the effort required in future suggested refactoring operations from software clustering results. Given that this section aims to close the loop between software clustering research and real-world application, the H2O AutoML framework also allows developers to incorporate REARRANGE within their workflow easily. Additional details are provided in Section 4.6.

This step answers the first research question, *RQ1: How can we provide reliable predictions of the effort needed to implement clustering results for refactoring?*

3.9. Model validation

This section discusses the quantitative and qualitative evaluation of the machine learning model developed in REARRANGE. By leveraging a combination of established metrics and assessments, we aim to provide a thorough understanding of the model's strengths, weaknesses, and overall reliability.

3.9.1. Quantitative evaluation

We performed validation on REARRANGE against sanity checks, baseline effort estimation models, and state-of-the-art software effort estimation models.

REARRANGE Validation Checks:

- Sanity Checks: Mean, Random
- Baseline Effort Estimation Models: Standardised Accuracy (SA), Variance Relative Error (RE*), Mean Absolute Error (MAE)
- State-of-the-Art Software Effort Estimation Models: COCOMOII, GeneticP, Software Maintenance, Fuzzy Logic

The above validations are explained in detail in Section 4.7. We use these validation checks to evaluate the accuracy of our prediction model, including standardised measures such as the Mean Absolute Error (MAE) and Standardised Accuracy (SA), which are recommended to compare prediction models [43,44]. Besides, we also utilise Variance Relative Error (RE*) suggested by Whigham et al. [19]. SA and RE* fulfil the requirements of a baseline effort estimation model discussed in the work by Whigham [19], which requires no tuning and is generally more accurate than a random guess or an estimate based purely on the distribution of the response variable. SA and RE* are cheap to apply and provide initial insights into the efficiency of REARRANGE. The full list of requirements can be found at the given link.⁷

Lastly, we compare REARRANGE with state-of-the-art software effort estimation models on the effort needed to perform the suggested refactoring operations. REARRANGE is compared with established software cost effort estimation studies for software development and bug-fixing, such as COCOMOII [45], Fuzzy Logic effort estimation [46], Software Maintenance effort estimation [47], and Genetic effort estimation [48]. To determine whether the estimates of an effort estimation technique are significantly better than another, the Wilcoxon Signed Rank Test [49] is carried out given that the Shapiro test [50] and Fig. 2 shows the data collected being non-normally distributed populations instead of the standard T-test. The following Null Hypothesis is tested: "The mean absolute errors provided by the effort estimation technique EET_s are not significantly lower than effort estimation technique EET_t for the data collected" [18]. If the distribution s is statistically significantly better (less) than t based on the Wilcoxon test, win_s and $loss_t$ are updated, else tie_s and tie_t is updated. On top of statistical significance, the effect size is measured as well using Vargha and Delaney's A_{12} non-parametric effect size measure [43,51] instead of Cohen's d due to our data being not normally distributed. All of the evaluated models and estimators have a large Vargha effect size.

To the best of our knowledge, there is limited work in estimating refactoring effort, and therefore, a lack of an open-source dataset to

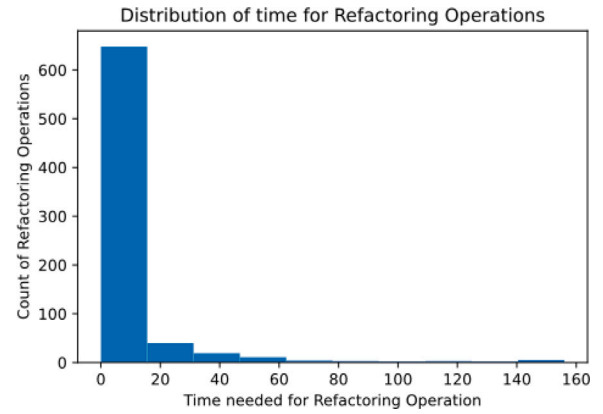


Fig. 2. Distribution of refactoring tasks in person-hours.

perform a comparative study. While these models may not be ideally suited for estimating refactoring effort due to the limited scope of refactoring within the broader context of software maintenance and development (including the efforts for development, testing, etc.), it is noteworthy that these models employ the number of lines of code as an input for effort estimation, much like our model. This serves as a valuable initial benchmark against which we can gauge the effectiveness of REARRANGE. For REARRANGE, our proposed approach focuses solely on the refactoring effort. The lack of a direct comparison also highlights the absence and need for a refactoring effort-focused effort estimation model.

3.9.2. Qualitative evaluation

Besides providing benchmarks of REARRANGE against other effort estimation techniques, we attempt to cross-validate REARRANGE's effort estimation with software developers to examine the trustworthiness of the proposed approach. By running REARRANGE on open-source Java projects, we can provide several recommended refactoring operations (based on clustering results) and their respective estimated effort required to developers via GitHub and Jira issues under suggested improvements. The responses from developers are detailed in Section 5.3. A survey involving software developers from various demographics is also carried out to obtain general feedback on effort estimates detailed in Section 5.4. This feedback would answer the second research question, *RQ2: How can we assess whether developers concur with our effort estimates in a real-world setting?*

3.10. Application

To close the gap between software clustering research and real-world application, developers could use REARRANGE to estimate the effort required for refactoring operations suggested by clustering algorithms.

Inputs: The proposed approach requires the following as inputs.

1. Source Code of the Project,
2. Preferred Software Clustering Algorithm (The most optimal algorithm can be established from E-SC4R [29]).

Outputs: Using DBeaver as an example, the output generated by REARRANGE Tool is shown in Table 6. The outputs shown are the time required for the refactoring operations suggested by the clustering algorithm.

4. Experimental setup

This section discusses the experiment setup and design in detail, including the collection of datasets, ground truth generation, building

⁷ <https://figshare.com/s/bff070c4b1a1ea9609a5>

Table 6
Sample Output for REARRANGE Tool.

Suggested refactoring for DBeaver	Predicted time in Person-hours
Move ByteOrderMark.java to util	3.214
Move FireBirdPlanTokenMatcher.java to io	8.0197

and training models, and benchmarking results. The scripts used in the experiment for each stage are publicly available.⁸

4.1. Experimental scoping

The experiment setup is to analyse if REARRANGE can estimate refactoring effort for the purpose of instilling confidence with respect to integrating REARRANGE into their daily development activities from the developers' point of view. In the context of estimating only code-based refactoring effort without taking into account downstream activities such as integration testing, we are attempting to build a machine learning model capable of estimating refactoring effort with a margin of error of 8 person-hours. We train REARRANGE with Software, Dependency, and Refactoring Features detailed in Section 3.5. REARRANGE's success is measured by the metrics MAE and RMSE to determine its efficacy.

4.2. Software systems

The study includes 25 Java-based projects from GitHub, selected based on the following search criteria.

1. The search parameters are set to identify Java-based projects at <https://github.com/topics/java>,
2. Projects are sorted by the number of stars,
3. Projects are manually chosen if they meet the following criteria,
 - The project should contain at least two major versions and ten minor versions in each major version,
 - The project should have been actively maintained over two years,
 - The project should have >1000 commits,
 - The project should have >100 stars on GitHub,
 - The project should have >100 forks on GitHub,
4. The most recent 20 releases of the selected projects will serve as the foundation for collecting refactoring data.

Upon filtering the available options based on these selection criteria, a pool of 40 projects emerged. Subsequently, through manual evaluation, we identified and selected 25 projects deemed to exhibit "high quality" attributes that align closely with the established criteria. The complete dataset can be found at the given link.⁹ Table 7 shows the list of software systems that were selected, where the columns *firstRelease* and *lastRelease* in the table indicate the software versions that we examined and used for our experiments. Selecting active and popular projects from GitHub would provide us with non-trivial projects and refactoring operations for our model to learn from. Projects with more than 100 stars or forks on GitHub were included to ensure a certain level of popularity and active engagement. Popular projects are more likely to undergo various refactoring operations, making them valuable for training the model. To ensure sufficient historical data for analysis, projects with at least two major versions or ten minor versions were selected. The model requires a significant amount of refactoring data to build reliable predictions, and projects with multiple major versions

and at least ten minor versions within each major version provide the necessary depth. These multiple versions allow for the examination of changes over time and help identify significant refactoring patterns. Without sufficient historical data, the model's ability to learn and predict refactoring operations accurately would be hindered.

When examining projects with more than 20 releases, it becomes imperative to consider the possibility of developers implementing a substantial overhaul of the system architecture. Such overhauls may involve significant additions to the software system, leading to considerable disruption in the generation of proxy ground truth, which serves as a crucial metric for evaluating refactoring efforts. For example, when comparing OkHttp 2.7.5 and OkHttp 5.0.0, the major feature additions in version 5.0.0 resulted in a drastic overhaul of the system architecture. This kind of overhaul could mask or distort the patterns that the model is trying to learn, making such projects unsuitable for inclusion in the study.

The inclusion criteria were designed to ensure that the selected projects would provide a rich dataset for examining refactoring operations in a collaborative, real-world setting, simulating realistic team dynamics. By focusing on popular, active projects with substantial version histories, the study maximises the chances of capturing meaningful refactoring patterns. Conversely, the exclusion criteria were aimed at eliminating projects that could potentially skew the results or provide insufficient data, ensuring that the model can make accurate and reliable predictions. Together, these criteria help create a robust dataset that aligns with the study's goals, enabling the model to learn from high-quality, relevant examples of refactoring operations across multiple software systems.

4.3. Identifying refactoring operations

Using Refactoring Miner, we identify all past refactoring operations and the time is taken for each chosen project.¹⁰ A sample of a recorded refactoring operation is given below for the operation "Extract Method" in the project *DropWizard*.

- type: Extract Method
- description: Extract private contains AuthAnnotation(annotations Annotation[])
- leftSideLocations
 - filePath: .../Auth/AuthDynamicFeature.java
 - startLine: 52, endline: 84
- rightSideLocations
 - filePath: .../auth/AuthDynamicFeature.java
 - startLine: 61, endline: 70

The following refactoring operations are recorded between each minor version: Extract Method, Inline Method, Rename Method, Move Method, Move Attribute, Pull Up Method, Pull Up Attribute, Push Down Method, Push Down Attribute, Extract Superclass, Extract Interface, Move Class, Rename Class, Extract and Move Method, Rename Package. In this study, we have narrowed our focus to the 15 primary refactoring operations identified by RefactoringMiner v3.0.4 [37].

4.4. Data processing and cleaning outliers

The Interquartile Range (IQR) methodology, as described by Wan et al. [52], is employed to identify outliers. The IQR is calculated as the difference between the third quartile (Q3) and the first quartile (Q1). Outliers are identified as data points that fall below the IQR minimum threshold (Q1 minus 1.5 times the IQR) or above the IQR maximum threshold (Q3 plus 1.5 times the IQR), given by Eq. (2),

⁸ <https://figshare.com/s/bff070c4b1a1ea9609a5>

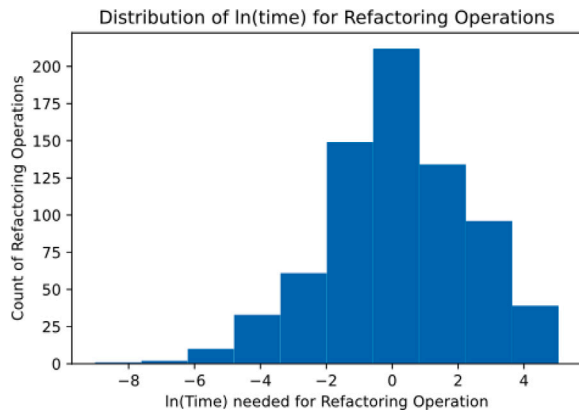
⁹ <https://figshare.com/s/0d24fa70b21104e7dcf1>

¹⁰ <https://figshare.com/s/f832c4592175974a6ec4>

Table 7

The list of selected projects and their detailed overview as of March 19, 2024.

Project Name	firstRelease	lastRelease	Domain	Number of Stars
Dbeaver	v21.0.0	v21.1.4	database solution	36 200
Apache Camel	v3.6.0	v3.11.1	data framework	5200
Redisson	v0.9.0	v3.16.1	database solution	22 500
Cxf	v3.3.5	v3.4.4	service framework	840
Apache Hadoop	v3.2.2	v3.3.1	database solution	14 230
Alluxio	v2.5.0	v2.6.1	data orchestration	6600
Beam	v2.27.0	v2.31.0	data processing	7400
Atmosphere	v2.6.0	v3.0.0	web framework	3700
Assertj-core	v3.16.0	v3.20.2	testing framework	2500
AxonFramework	v4.4.4	v4.5.3	microservice framework	3200
Byte-buddy	v1.11.1	v1.11.12	virtual machine solution	5900
Cas	v6.3.1	v6.4.0	security framework	10 600
Karaf	v4.2.4	v4.3.2	virtual machine solution	640
Storm	v1.1.0	v2.2.0	computing solution	6500
Spring-Framework	v5.3.0	v5.3.9	java framework	54 600
Spring-Security	v5.4.4	v5.6.0	java framework	8300
Apache Cassandra	v3.11.111	v4.0.0	database solution	8400
OkHttp	v4.5.0	v5.0.0	web framework	45 100
Openapi-Generator	v4.2.2	v5.2.0	API framework	19 300
Testcontainers-Java	v1.1.0	v1.2.1	testing framework	7700
Traccar	v4.2.0	v4.13.0	gps solution	4700
OrientDB	v3.1.3	v3.2.0	database solution	4700
Spotbugs	v4.0.5	v4.3.0	testing framework	3300
Tika	v1.22.0	v2.0.0	text parser solution	2100
Rest-Assured	v3.3.0	v4.4.0	microservice framework	6700
Guacamole-Client	v0.9.13	v1.3.0	java framework	1300
Eclim	v2.3.2	v2.8.0	source code editor	1000
Hawtio	v2.11.0	v2.13.5	java framework	1400
Pxb1988-Dex2jar	v0.9.14	v0.9.9	java framework	11 700
Web3j	v4.5.2	v4.6.1	java framework	4900

**Fig. 3.** Distribution of refactoring tasks after applying $\ln x$ function.

Eq. (3) and Eq. (4). Data points falling outside the range are classified as outliers. The chosen outlier detection method is applied to the dataset, and outliers are subsequently removed. This method helps in focusing on the central portion of the data distribution and identifying values that significantly deviate from the overall pattern, aiding in detecting potential anomalies or extreme observations within the dataset. While there is a potential for information loss when removing outliers, these outliers lying outside the interquartile range are attributed to exceptionally long commits, suspected to stem from developer inactivity, such as breaks or holidays.

$$IQR = Q3 - Q1 \quad (2)$$

$$IQR_{MinimumThreshold} = Q1 - 1.5 * IQR \quad (3)$$

$$IQR_{MaximumThreshold} = Q3 + 1.5 * IQR \quad (4)$$

4.5. Data transformation

Fig. 2 shows the time distribution in person-hours for all the identified refactoring operations computed based on Eq. (1). The effort distribution is an uneven “long-tail” distribution, where a significant portion of refactoring operations can be completed relatively quickly. Refactoring operations that require a significant amount of time to complete are rare and harder to predict, but these operations are the most destructive to the timeline of a project. These outliers are one of the main reasons developers are hesitant to commit to refactoring tasks [15]. The data extracted are normalised and transformed to accommodate these outliers.

We found that the distribution of the refactoring effort closely resembles the bug-fixing effort reported in the work by Zhang et al. [53]. To factor in these outliers in model training, we adopt the technique used by Jorgensen and Zhang [53–55], which is to apply a natural logarithm, $\ln x$ function, to the ground truth effort estimation data, x , to convert the distribution to a normal distribution. The distribution of the time taken for refactoring tasks after applying the $\ln x$ function is shown in Fig. 3. The prediction output of $\ln(\text{person} - \text{hours})$ will then be converted via an exponential function $\exp(\ln(\text{person} - \text{hours}))$ back to $\text{person} - \text{hours}$ for developers using REARRANGE.

4.6. Model training

The train, test, and validation data sets are split with a ratio of 6:2:2, respectively, with seed = 16. The training data is then fitted to a regression model.¹¹ Testing of REARRANGE is validated with k-folds of 3. Although the model is trained with input data of $\ln(\text{ActualEffort})$, and provides an output of $\ln(\text{PredictedEffort})$ the results in the subsequent sections are being reported by using PredictedEffort given by $e^{\ln(\text{PredictedEffort})}$, which are converted back to person-hours.

¹¹ <https://figshare.com/s/72b234bc18bcd894528>

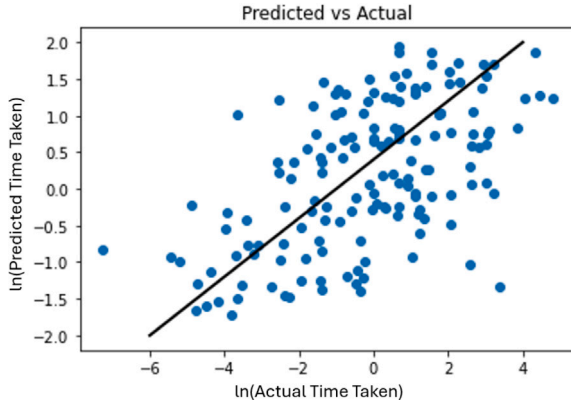


Fig. 4. Regression predicted vs actual time taken, in $\ln x$.

4.7. Model evaluation

The accuracy of the models built in Section 3.8 are then subsequently subjected to validation using the following metrics.

4.7.1. Baseline effort estimation models

The evaluation criteria when evaluating REARRANGE with MAE, SA, and RE^* are given in order below [18]:

$$MAE = \frac{1}{N} \sum_{i=1}^N |GroundTruth_i - PredictedEffort| \quad (5)$$

MAE is given where N is the number of projects during evaluation, and GroundTruth and PredictedEffort are the actual and predicted effort for refactoring, respectively, for project i .

$$SA = (1 - \frac{PredictedEffort}{MAE_{guess}}) \times 100 \quad (6)$$

SA is given where $MAE_{EffortEstimated}$ is the MAE of the Effort Estimation Techniques in Section 4.7 with REARRANGE whereas MAE_{guess} is the MAE of random guesses. SA represents how much better the Effort Estimation Technique is than random guessing, where values close to 0 would mean that the Effort Estimation Technique is no better than random guessing.

$$RE^* = \frac{var(PredictedEffort - ActualEffort)}{var(ActualEffort)} \quad (7)$$

RE^* gives a score of 1 to a prediction model with zero variance, a score of less than 1 to a “good” Effort Estimation Technique, and a score of greater than 1 to a “bad” Effort Estimation Technique [18,19]. Variance is calculated by taking the average of squared deviations from the mean.

4.7.2. State-of-the-art software effort estimation models

The accuracy of the models built in Section 3.8 is compared to existing software cost effort estimation methods for software development and bug-fixing, including COCOMOII [45], Software Maintenance effort estimation [46], Genetic effort estimation [47], and Fuzzy Logic effort estimation [48]. The numbers shown in Section 4.7.2 represent the default values for the equations prescribed by the relevant literature. These values are tailored for estimating software development effort using Lines of Code as a metric. when there is no prior knowledge for fine-tuning. In order to establish a reliable baseline, we have decided to follow the default values provided by the relevant literature.

$$COCOMOII = 2.94 \times KLoC \times 176 \quad (8)$$

$$SoftwareMaintenance = -40 + 6.56 \times LOC \quad (9)$$

$$Genetic = ((1.75992 \times KLoC) - (4.56 \times 0.001 \times (KLoC^2))) \times 176 \quad (10)$$

$$Fuzzy = \frac{((4.2KLoC)^{0.665}) + (24KLoC^{0.665}) + (7.8KLoC)^{0.665}}{(1 + 4 + 1)176} \quad (11)$$

5. Results and discussion

This section discusses the results obtained from the experimental study, including the model’s accuracy and the model’s benchmark against other software cost effort estimation studies.

5.1. Chosen machine learning model

The target variable in the regression models is $\ln$ (time taken for refactoring effort). The $\ln$ (predicted time taken) vs. $\ln$ (actual time taken) is shown in Fig. 4. The figure shows a reasonably good fit and a strong correlation between the predictions of the model and its actual results, where most of the points lie close to the fitted line. Although there are some outliers, we found that these are typically data points with extremely high refactoring efforts (40 h or more, which means refactoring operations that take more than one week) or extremely low refactoring efforts (one hour or less).

After prediction, the target variable is converted back to person-hours for comparison. Based on H2O’s search parameters, the Gradient Boosting Machine is deemed the best based on Root Mean Squared Error (RMSE) with an RMSE person-hours of 15.213. REARRANGE estimates the refactoring effort with a Mean Absolute Error of 5.47.

5.2. Benchmarking of REARRANGE

The comparison of SA and RE^* in Table 8 of REARRANGE against Sanity Check methods (random and mean) shows that REARRANGE is more accurate than random guessing or an estimate based solely on the distribution of the response variable [18]. REARRANGE performs much better than random and mean estimates, with the mean being closest in SA. Using the mean can be misinterpreted as a good estimate due to the distribution of the refactoring data, as observed in Fig. 2. However, the mean would be costly when predicting the effort for large refactoring operations, as seen in the difference in MAE and RMSE in Table 11.

The comparison of RMSE of REARRANGE against other software cost effort estimation models is given in Table 11. The RMSE results show that the existing software cost effort estimation models are not good estimators of the refactoring effort. The MAE (Mean Absolute Error) of 5.47 for REARRANGE is within an acceptable error of 1 working day of 8 person-hours. Although REARRANGE may sometimes overestimate the time required for smaller refactoring operations, it can accurately identify and predict large and complex refactoring operations that are incredibly disruptive to developers. Similarly, existing software effort estimation models significantly overestimate the time required compared to REARRANGE, resulting in high RMSE and MAE values.

By comparing MAE between REARRANGE (5.47) and Genetic effort estimation (453.31) as the next best alternative from Table 11, it is evident that REARRANGE performs better at estimating refactoring effort. The significant difference in MAE is due to Genetic effort estimation being a software effort estimation model that considers the time to develop a complete software. The lack of a direct comparison further highlights the need for a refactoring effort-focused effort estimation model.

Table 9 shows the win-tie-loss outcome of the Wilcoxon test between MAE distributions where the subjected effort estimation technique scored a p -value < 0.001 (win), p -value > 0.99 (loss), and $0.001 \leq p$ -value ≤ 0.99 (tie) [18]. Similar to the results based on MAE,

Table 8
SA and RE* obtained by Sanity Check and State-of-the-Art Methods.

Type	Effort estimation model	SA	RE*
Sanity Check	Mean	75.87	1.0
Sanity Check	Random	0	11.56
Basic	REARRANGE	93.627	0.909
State-of-the-art	COCOMOII	−1038.39	113 484.19
State-of-the-art	GeneticP	−477.72	20 599.25
State-of-the-art	SoftwareMaintenance	−14315.26	18233861.80
State-of-the-art	Fuzzy Logic	−1088.11	21 978.24

Table 9
Wilcoxon test results of REARRANGE and other effort estimation models.

Effort estimation model	Estimator	win	loss	tie
REARRANGE	Mean	1	0	0
REARRANGE	Random	1	0	0
COCOMOII	Mean	0	1	0
COCOMOII	Random	0	0	1
GeneticP	Mean	0	1	0
GeneticP	Random	0	0	1
SoftwareMaintenance	Mean	0	1	0
SoftwareMaintenance	Random	0	1	0
Fuzzy	Mean	0	1	0
Fuzzy	Random	0	1	0

Table 10
Vargha and Delaney effect size of REARRANGE and other effort estimation models.

Effort estimation model	Estimator	Effect size
REARRANGE	Mean, Random	Large
COCOMOII	Mean, Random	Large
GeneticP	Mean, Random	Large
SoftwareMaintenance	Mean, Random	Large
Fuzzy	Mean, Random	Large

Table 11
Benchmark results for REARRANGE (person-hour)

Effort estimation model	R2	RMSE	MAE
Mean	−0.974	21.37	18.93
Random	−35.76	92.246	78.46
REARRANGE	0.538	15.213	5.47
COCOMOII	−116903	5201.57	893.22
GeneticP	−21469.5	2229.15	453.31
SoftwareMaintenance	−18786447	65 938.96	11 310.71
Fuzzy	−25722.63	2439.97	932.23

REARRANGE outperforms the other effort estimation techniques with a large effect size (see Table 10).

Answers to RQ1: REARRANGE outperforms baseline effort estimation models and state-of-the-art software effort estimation models when predicting specifically refactoring effort. The absence of a direct comparison underscores the current state of the available options, making REARRANGE one of the more reliable approaches to estimate refactoring effort. REARRANGE estimates the refactoring effort with a Mean Absolute Error of 5.47 person-hours, within an acceptable error range of 8 person-hours per day. As such, REARRANGE is a dependable approach for us to obtain reliable predictions to implement clustering results, especially when estimating large refactoring efforts. The reliability of REARRANGE becomes even more evident without other established and reliable methodologies.

5.3. Validation from project developers

Next, we proceed to obtain feedback from the developers of the chosen test subjects on the proposed refactoring operations and the

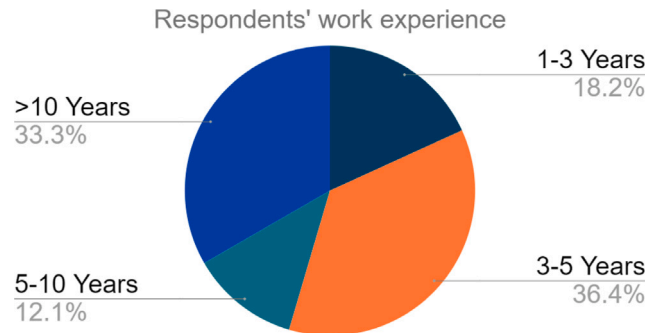


Fig. 5. Respondents' work experience.

estimated efforts. This is done by filing issue reports on the relevant GitHub repository or Jira.

Fig. 5 shows the demographics of respondents' work experience. We attempt to obtain feedback from participants with more than one year of software development experience. Among the total 53 suggested refactoring operations and the estimated effort required, 27 received a positive response from current contributing developers, with only one such operation receiving a negative response, as shown in Table 12. The rest of the suggested refactoring operations are still pending a response from developers (Pending), or they cannot provide an estimated effort for the required refactoring (N/A).

Among the favourable responses received from developers, one notable instance is observed in Dropwizard Issue #5402, where the developer expressed "If you look at my Pull Request (PR) history for Dropwizard you will see it is pretty much 100% refactoring (depending on your definition...). I've probably spent about 50–100 h on it. So if you run your algorithm on the before/after of those PRs you can check your maths!". These observations align closely with the effort estimates suggested by REARRANGE, which fall within the range of 32–60 h.

Overall, the positive sentiments from developers show that REARRANGE is able to provide reasonable estimates relevant to software developers.

5.4. Validation from general developers

To further validate the effectiveness of REARRANGE to a wider group of audience, we carried out a survey¹² involving software developers from various demographics based on the survey guidelines provided by Jefferson et al. [56]. Our survey has obtained ethical approval from Monash University with the designated approval number 36892. We conducted a comprehensive evaluation of REARRANGE effort estimates by engaging academics and industry practitioners. Through a survey-based approach, we sought their expert opinions in aligning the effort estimates generated by REARRANGE.

¹² <https://forms.office.com/r/XeFh7UUGfT>

Table 12

Developer feedback from issues raised in the project Github or Jira.

Project Name	Issue ID	No. of Suggestions	Positive Response	Negative Response	Status
Dropwizard	5402	4	4	0	Closed
Redisson	4363	5	0	0	Closed
Camel	18230	7	0	0	Closed
Okhttp	7355	17	17	0	Closed
Flink	28252	3	0	0	Closed
Atmosphere	2479	3	0	0	Closed
Axon Framework	2271	3	3	0	Closed
Hawtio	2752	2	0	0	Closed
Java-tron	4500	2	1	1	Closed
OrientDB	9838	3	2	1	Closed
Spotbugs	2111	3	0	0	Closed
Spring Framework	28749	1	0	0	Closed

5.4.1. Survey research objective

The questionnaire is centred around two specific tasks, targeting a thorough assessment of REARRANGE's utility.

1. In the initial task, participants were asked to evaluate their feasibility in performing the recommended refactoring operation within the designated timeframe.
2. In the subsequent task, participants were asked about suggested revisions for the allocated timeframe (more or less time required) if they determined they were unable to complete the suggested refactoring operation within the given timeframe.

5.4.2. Study plan

Throughout the research project, the primary focus has been on assessing the utility of REARRANGE for developers, with particular attention to the accuracy of its effort estimations. It is imperative for REARRANGE to minimise over-predictions and under-predictions to ensure its effectiveness, motivating the implementation of this survey.

5.4.3. Identify population

The survey targets developers within the general public who may participate in refactoring activities.

5.4.4. Sampling plan - Subject participants

The sample for this survey is selected randomly, employing a probabilistic approach. Potential respondents participate in a self-selection process by volunteering themselves. To ensure that participants possess relevant experience in software development encompassing the entire lifecycle, including refactoring, we employ a filtering criterion mandating a minimum of one year's experience in developing software systems. We have obtained responses from 22 participants (4 Ph.D. students and 18 professional software developers). 45% of the participants have over five years of experience, with 33.3% of them consisting of senior developers with >10 years of experience.

Our evaluation encompassed five open-source software projects: AxonFramework, Apache Camel, OkHttp, Apache FLink, and OrientDB. For each software, two to four refactoring operations are randomly chosen to be evaluated. REARRANGE will generate all of the refactoring operations to convert the current package structure to the recommended structure suggested by the most optimum clustering algorithm. Recognising the significant time commitment required for participants to analyse all thoroughly suggested refactoring operations derived from clustering results, only a subset of operations was randomly selected for evaluation. This approach ensured participants could effectively evaluate REARRANGE's effectiveness and complete the questionnaire within a manageable timeframe.

5.4.5. Instrument design - Questionnaire

This survey is self-administered through web-based questionnaire pages, where respondents input their data. The questionnaire contains three parts:

- Questions to collect practitioners' background information, such as their development experience.
- Questions to obtain practitioners' familiarity with software refactoring
- Questions to evaluate the soundness of REARRANGE's effort estimation.

Part 1: The questions from the first part include:

1. Please provide your email address. (Kindly leave blank if you wish your response to be anonymised.)
2. Which company are you working for?
3. How many years of experience do you have in developing software systems?

Question 3 serve the purpose of filtering respondents who possess foundational knowledge in software development, allowing a refined analysis of the collected data based on distinct levels of development experience.

Part 2: The questions from the second part include:

1. How important do you think refactoring is for the overall benefit of long-term software quality?

Question 4 serve the purpose of gauging respondent's familiarity and sentiment towards refactoring operations and efforts.

Part 3: The questions in the third part are formulated in the following manner: "If a refactoring task is given to you to shift the following classes to a new package, can you complete it within the suggested timeframe?". Each question encompasses a subset of suggested refactoring operations derived from the clustering results of the respective software system. Specific details of each question is made publicly available.¹³

5.4.6. Instrument validation

In the initial phase of our survey development, we engaged a small cohort of 5 pilot testers, comprising of peers who are postgraduate students. Our pilot testers encountered challenges when prompted to provide estimations of person-hours required for refactoring activities. They preferred receiving additional assumptions and context to facilitate their estimations. This insight led us to incorporate model estimates into the survey, enabling participants to justify their ability to complete the task within the specified time frame based on provided assumptions and contextual information. Another feedback gathered from this pilot test is that there is little significance in visualising the impacted refactoring operations. Furthermore, it became evident that including unnecessary code elements would overwhelm participants with information. Consequently, in the subsequent refinement of the survey instrument, we presented only the pertinent or affected classes to participants. We have added the discussion of the pilot study in the revised manuscript.

¹³ <https://figshare.com/s/60429a89514493aaddb7>

Participants were given a concise, 5-minute guide containing explanations and motivations behind REARRANGE. The expected responses were intentionally withheld from the participants to ensure impartial outcomes, mitigating the potential for biased results.

5.4.7. Participant recruitment

Participants for this survey are recruited through self-recruitment. Additionally, ancillary documents, including an explanatory statement and a consent form, are provided to participants. The explanatory statement outlines details regarding the research project, the storage of survey data, potential benefits and risks, as well as confidentiality measures.

5.4.8. Response management

Because of voluntary participation, the participation rate reaches 100%. Given the anonymous, web-based format of the survey, no measures are taken in the event of non-responses.

5.4.9. Data analysis

Table 13 displays survey results indicating that REARRANGE consistently delivers accurate estimates in 93.6% of cases, demonstrating a 2.1% overestimation and 4.3% underestimation. These findings underscore the widespread consensus among developers regarding the reliability of REARRANGE's estimates. In the few cases where the developers felt that the estimated time (from REARRANGE) was insufficient for the refactoring operation to be completed, they usually involved larger projects with >2000 KLoC, such as ApacheCamel and OkHttp. Responses from participants indicated that large projects tend to have dependencies between classes that are very complex, which requires more effort to understand the source code and ensure that the refactoring operation does not break other functionalities. Conversely, in smaller projects with <300KLoC, such as AxonFramework, the response from the developers indicated that the refactoring operations are much more straightforward with little to no complexities and only require a fraction of the allocated time. The results from this survey further shows REARRANGE's ability in estimating refactoring effort, except for some outliers.

In general, the majority of incorrect estimates generated by REARRANGE tend to be undereffort estimations, where developers require more time for the suggested refactoring operations. This suggests a potential opportunity to fine-tune the REARRANGE model by providing slightly larger effort estimates, accommodating a broader range of developers in future scenarios.

Among the refactoring operations examined, the task involving *ErrorHandlerReifier.java* exhibited the highest proportion of undereffort estimations, as developers expressed a need for additional time to execute the suggested refactoring operation. This observation can likely be attributed to the complex nature of *ErrorHandlerReifier.java*, which encompasses 26 dependencies within its internal packages. The intricate interdependencies make the refactoring process significantly more intricate, necessitating meticulous attention to prevent disruptions to existing functionality.

5.4.10. Reporting

The survey and ancillary documents are made publicly available and accessible.¹⁴ Due to the small sample size in this survey, the survey results should be considered as a general consensus among developers regarding REARRANGE. Further validation through an expanded deep dive survey or use case analysis could enhance the justification for REARRANGE.

5.5. Summary of experimental results

REARRANGE can inform developers of potentially complex and time-consuming refactoring operations, which would give developers a better gauge of the time and effort needed to carry out a specific refactoring operation.

From a **practitioner's perspective**, REARRANGE can provide each of the suggested refactoring operations with an estimated effort. Developers can then choose which refactoring operations to prioritise and integrate refactoring into their workflow. Using Agile development sprints as an example, developers can attend to and select refactoring tasks based on their sprint duration. For example, for 1-week sprints, they can select 30-hour development tasks and 10-hour refactoring tasks that add up to 40 person-hours.

From a **research perspective**, the REARRANGE approach is an end-to-end pipeline to estimate the refactoring effort. The same approach can be used to curate or create an open-source refactoring effort dataset to improve the replicability and reliability of effort estimation research. On top of that, we have also shown that the proposed approach is generic and flexible, capable of supplementing existing software clustering research, paving the way to close the loop for software remodularisation.

Answers to RQ2: Obtaining precise effort estimations of refactoring efforts directly from developers poses challenges. Nevertheless, the overall sentiment developers highlight the value our estimates offer as a foundational reference and guidance to incorporate refactoring endeavours into their daily workflow. While feedback varies, a predominant majority of developers, including both general developers and those involved in specific projects, express a positive outlook on the potential of REARRANGE.

6. Threats to validity

In this section, we identify and discuss potential threats to the validity of our research findings, aiming to ensure the reliability and credibility of our conclusions.

6.1. Threats to validity - REARRANGE's approach

Based on the classification schema by Runeson et al. [57], **Construct Validity** in our project refers to whether all the relevant parameters for regression models have been tested to obtain the best possible model and hyperparameters to model the relationship between features and the target variable. We have decided to use H2O's AutoML [42] pipeline to search and deduce the best possible model and hyperparameters to mitigate this risk.

Internal Validity The internal metrics used in the evaluation can affect the validity of the results. To address this concern, we adopted three renowned and extensively employed metrics [33,34,36]. Another potential threat to validity arises from the participant selection for the survey, as feedback from various types of participants may impact the study outcomes. To alleviate this concern, we extended invitations to both academics (Ph.D. students) and industrial developers, ensuring a diverse range of perspectives. Furthermore, in order to mitigate the risk associated with participant selection, we exclusively recruited individuals with a minimum of one year of experience in software development. This criterion ensured a certain level of familiarity with the software development lifecycle, encompassing refactoring practices.

Concerning **External Validity**, we acknowledge that there is a risk that the approach may not perform as well for specific projects that fall outside the domain of our selected projects. While our research primarily focuses on estimating general refactoring efforts, we recognise the significance of continually improving our approach to addressing the

¹⁴ <https://forms.office.com/r/gaAzcWx2VB>

Table 13
Results from qualitative evaluation.

Project name	Source package	Destination package	Class involved	Over estimate %	Good estimate %	Under estimate %
AxonFramework	messaging	handler	HandlerAttributes.java	4.8	95.2	0.0
AxonFramework	messaging	handler	AnnotatedHandlerAttributes.java	5.6	88.9	5.6
AxonFramework	messaging	handler	AnnotatedMessageHandlingMember.java	11.1	88.9	0.0
Apache Camel	errorhandler	<i>new</i>	ErrorHandlerRefReifier.java	0.0	94.7	5.3
Apache Camel	errorhandler	<i>new</i>	ErrorHandlerReifier.java	0.0	88.2	11.8
Apache Camel	errorhandler	<i>new</i>	NoErrorHandlerRefier.java	0.0	94.1	5.9
Apache Camel	errorhandler	<i>new</i>	DefaultErrorHandlerRefier.java	0.0	100.0	0.0
OkHttp	okhttp3	<i>new</i>	AccessHeaders.java	0.0	90.5	9.5
OkHttp	okhttp3	<i>new</i>	CustomCipherSuites.java	0.0	94.4	5.6
OkHttp	okhttp3	<i>new</i>	HpackJsonUtil.java	0.0	94.4	5.6
Apache FLink	tasks	coordinator	SubtaskCheckpointCoordinator.java	4.8	95.2	0.0
Apache FLink	tasks	coordinator	SubtaskCheckpointCoordinatorImpl.java	0.0	94.4	5.6
OrientDB	security	osecurity	OSecurityRemote.java	0.0	95.5	4.5
OrientDB	security	osecurity	OSecurity.java	5.3	94.7	0.0
OrientDB	security	osecurity	OSecurityShared.java	0.0	94.7	5.3

randomisation factor and bias when selecting projects as subjects. To mitigate this risk, we selected a reasonable representation of software subjects from different domains (including HTTP clients, distributed processing engines, to virtual distributed file systems) and sizes¹⁵ (ranging from Java-tron with 13KLoC to Apache Hadoop with 1774KLoC), and complexity. This study considers aggregated refactorings due to limited refactoring data. Splitting into individual refactoring types will require many past refactoring effort datapoints. These can be mitigated by replicated on a larger scale and continuously add new refactoring data to the current dataset. Refactoring data from projects written in different languages would be valuable to verify the current findings. The refactoring estimate here only provides an estimate of the process. Refactoring sometimes leads to additional downstream activities, such as integration testing, not captured in the same commits. We acknowledge that the estimates provided may vary depending on the experience and proficiency of the developers. REARRANGE aims to benefit developers new to the project to prioritise important refactoring tasks. For companies/projects with a larger proportion of senior developers and enough data points (past commits), retraining the model with their respective version control/commit histories instead of the data source would allow it to better recognise the effort estimation for refactoring in such scenarios.

6.2. Threats to validity - REARRANGE's survey

In assessing the validity of the survey conducted for REARRANGE, there are several potential threats that should be taken into consideration, particularly regarding the instrument and methodology employed.

With regards to **Instrument Validity**, the primary threat to the validity of the survey lies in its reliance on a relatively small sample size. Consequently, the survey results should be interpreted cautiously, representing more of a general consensus among developers regarding REARRANGE rather than comprehensive insights. While efforts were made to ensure representativeness within the sample, the inherent limitations associated with a small sample size could introduce biases and limit the generalisability of the findings. To mitigate this risk, we recruited participants from diverse demographic backgrounds and varying levels of experience in software development. By ensuring representation from different demographic groups, including individuals with varied levels of expertise and experience, we aimed to capture a broader spectrum of perspectives and insights. This approach helped mitigate the risk of bias and ensured that the survey results were more representative of the broader developer community.

Concerning **Methodological Validity**, the methodology utilised for conducting the survey also presents potential threats to its validity. The

survey's design and administration may have inadvertently influenced respondents' perceptions and responses. To mitigate this risk, the survey instrument itself was carefully designed to encompass questions addressing different aspects of the topic under investigation. By assuring respondents of the anonymity and confidentiality of their responses, this encourages honest and critical feedback, mitigating the risk of respondents providing socially desirable responses or altering their responses based on perceived expectations. Moreover, efforts were made to conduct the survey in a manner that maximised participation and engagement among respondents through pilot testing. This involved employing clear and concise language in survey questions, providing adequate context for respondents to understand the purpose of the study, and ensuring that the survey platform was user-friendly and accessible across different devices. These measures aimed to minimise potential barriers to participation and enhance the quality of responses obtained.

7. Related work

There is a significant gap in research in refactoring effort estimation. This section reviews papers related to our study mainly from software clustering and refactoring research.

Identify Upper and Lower Bound of Refactoring Changes: The work by Paixao et al. [15] considers the lower and upper bounds of refactoring change acceptable by developers for the change to occur. This is done by analysing the project's past activity, which documents all previous disruptions (DisMoJo — the number of Move Join operations to change from one software structure to another) to the software structure and records the lower and upper bounds. The lower bound is helpful as an estimate of the smallest possible disruption that developers can accept if the refactoring process is carried out routinely. The upper bound helps estimate developers' largest possible disruption tolerance for a single refactoring exercise. The experiment was carried out on 10 open-source software with 233 releases. However, the estimated effort given by the authors provided a relatively abstract effort by comparing changes based on the move and join operations from past releases. This may not provide the confidence needed for developers as the effort for each refactoring may differ in complexity, such as the total lines of code affected and the number of dependencies of the affected class to other classes. Besides that, the estimated effort would differ based on the number of active developers, the average contribution per developer, and each developer's expertise. Given the final clustering result by a clustering algorithm, there is a need to provide an estimated effort in relative hours to help developers in the refactoring process.

Quantifying the Benefits of Refactoring: The work by Martini et al. [39] quantifies the benefits of refactoring by closely following the development team's workflow and recording the developer hours required to maintain the system before and after refactoring.

¹⁵ <https://figshare.com/s/b6fb34ded2028dd2a4fb>

By focusing on the interest calculation related to the effort required for development and maintenance to refactor an existing component to repay technical debt, the benefits of refactoring are shown in the reduced hours in maintaining the system. The effort gained by refactoring is given by $(\text{old development effort} - \text{new development effort}) \times \text{time} + (\text{old maintenance effort} - \text{new maintenance effort}) \times \text{time} - \text{refactoring effort}$. However, the case study only quantifies refactoring benefits based on effort reduction in the new maintenance effort after refactoring activities. It requires collaboration with developers over a long period to track and record the maintenance effort before and after refactoring. The overall process suggested by the authors may not apply to many software systems in a real-world setting.

The work by Li et al. [58] provided a systematic mapping study on technical debt (TD) and technical debt management (TDM). Refactoring is a common approach to repay technical debt by improving the internal quality of a software system without altering its external behaviour. It is one of the most used approaches for TD repayment, as it directly addresses code and design TD. Quantifying the benefits of refactoring is challenging because it can be difficult to predict which TD items will have the highest future cost and to estimate the cost and benefit of refactoring accurately. There is a gap in the current literature to develop effective measurement models. This highlights the need for empirical studies to inform stakeholders about the cost and benefits of managing TD through refactoring. Such studies can help demonstrate the success or failure of TDM approaches, including refactoring in real-world scenarios.

A study focusing on the impact of object-oriented code refactoring on software quality was conducted by Kaur et al. highlighting the general findings of 142 primary studies published from 2003 till December 2017 [12]. The authors found that refactoring generally leads to an improvement in software quality attributes such as coupling, size, and encapsulation. However, the effects on cohesion, complexity, and inheritance are inconsistent or unchanged. From the perspective of industry versus academia for software refactoring, the positive impact of refactoring on software quality is more frequently reported in academic settings than in industrial ones, suggesting a need for more industrial validation. The primary studies also collectively indicated the insufficient availability of tool support. There is a lack of updated tools to predict or assess the benefits of refactoring activities, which hinders developers from adopting refactoring practices in the software industry. The development of new tools and IDE plugins could facilitate the measurement of refactoring's impact on software quality.

The work by Aniche et al. [59] investigates the effectiveness of machine learning algorithms in predicting software refactorings, with models trained on over two million refactorings from real-world projects showing high accuracy in recommending specific refactoring operations. This established a positive correlation between refactoring operations and code quality metrics, indicating that refactoring should be a primary concern for software developers. The team discusses the challenges developers face in deciding when and what to refactor, emphasising that refactoring activities come with costs and should not be undertaken without careful consideration.

The study by Kadar et al. [60] discusses the gap between theoretical aspects of code refactoring and its practical application by developers. The authors suggest that developers often refactor code differently than what theory prescribes. An extensive dataset is provided, containing source code metrics and applied refactorings across several releases of 7 open-source systems. This dataset aims to facilitate empirical research on how refactorings are performed in practice. Early results from analysing the dataset show that classes with lower maintainability are more frequently refactored, leading to significant improvements in complexity, size, coupling, and clone metrics, although a decrease in comment-related metrics was observed.

Refactored code elements (classes and methods) typically have lower maintainability and higher complexity, size, and coupling metrics compared to non-refactored elements, indicating that these factors

are likely considered when identifying refactoring opportunities. The study by Hegedus et al. [61] suggests that refactoring is often applied to code with poor maintainability to improve its internal structure, as refactored elements show a significant increase in maintainability post-refactoring. After refactoring, the affected code elements exhibit controlled growth or reduction in key metrics such as size, complexity, and coupling, implying that refactoring helps manage these quality attributes effectively.

Effort Estimation: Deeb et al. were able to predict a relative level of refactoring effort for the next release using code metrics of previous releases [45]. The team found moderate correlations between refactoring efforts and project size, code complexity, and architecture smells, successfully predicting higher refactoring costs using machine learning techniques by applying the COCOMO II:2000 model on 33 Apache Java projects.

Moulla et al. estimate software development costs by considering the number of commits per contributor per release, active days per contributor in each release, and the number of lines of code added per contributor in each release [62]. Contributors are then classified into Occasional (active < 1/3 of the release duration), Part-time (active < 2/3 of the release duration) and Full-time (active > 2/3 of the release duration). This allows for a more accurate calculation of the total number of person-hours required to develop a feature or release, where the effort estimates for projects with a mix of part-time and occasional contributors may require additional time buffers. However, the effort estimates only target software effort estimation instead of refactoring effort. Considering the scarcity of research on estimating refactoring effort, a relevant source of insights can be derived from well-established studies in software cost estimation, particularly those on software development and bug-fixing. These studies are dedicated to addressing effort estimation challenges within their respective domains.

The work by Mittal et al. introduces a novel approach to estimating software development effort by representing project size in KLoC as a Fuzzy number and employing fuzzy logic [46]. This is done by fine-tuning the COCOMO II model using MATLAB and evaluating its performance with real-world software project data. Hayes proposed a similar approach with a different methodology to estimate the effort required for adaptive software maintenance in person-hours [47]. Several metrics, including lines of code changed and the number of operators changed, were identified as highly correlated with maintenance effort. The regression models built using these metrics proved effective in accurately predicting adaptive maintenance efforts, offering valuable insights for managers and maintainers.

The work by Alaa and Al-Afeef tries to develop a precise mathematical model that can establish a reliable connection between software project effort or cost and its various cost drivers by utilising Genetic Programming (GP) to create a software cost model that incorporates factors such as lines of code and development methodology [48]. The GP-based model demonstrated strong estimation capabilities, outperforming existing models in the literature, particularly when applied to estimating the effort of National Aeronautics and Space Administration (NASA) software projects.

The work by Tawosi et al. [63] presents a new approach for Story Point (SP) estimation in agile software development using latent Dirichlet allocation (LDA) and hierarchical clustering to analyse textual features of software issues. The proposed method, LHC-SE, aims to reduce variability in software engineering data by clustering issues into groups based on topic similarities, which is expected to enhance the accuracy of SP estimations. The approach was evaluated on a dataset of 26 open-source projects with over 31,000 issues. The results indicated that while the method's performance was comparable to state-of-the-art techniques, it did not significantly outperform more naive estimators in all cases. The paper encourages future work to develop alternative strategies for SP estimation.

Software engineering relies on analogy-centric effort estimation, where historical data is used to estimate the effort required for new

software projects. This approach leverages existing project histories to predict development effort accurately. Existing Software Effort Estimation (SEE) methodologies face challenges related to correctness, bias, and subjectivity. Kumar [64] proposed an analogy-based SEE (AA-SEE) technique that combines hybrid optimisation and ML methods. It introduces a multi-swarm coyote optimisation (MSCO) algorithm to fine-tune hyperparameters for ML models. Additionally, it employs a teaching-learning-based recurrent neural network (TL-RNN) for more accurate effort estimation. Evaluation using standard datasets demonstrates its performance compared to existing state-of-the-art methodologies in terms of accuracy and various error metrics.

Khan [65] and team discusses the use of meta-heuristic algorithms, specifically Grey Wolf Optimizer (GWO) and StrawBerry (SB), to create a logical and acceptable parametric model for software effort estimation. It presents a Deep Neural Network (DNN) model optimised by these meta-heuristic algorithms for initial weights and learning rate selection, which produced better results compared to existing methods. This emphasises the importance of accurate software effort estimation in managing software projects effectively, as it involves predicting project cost, effort in man-hours, and time duration to complete a project. Accurate estimation is crucial for project success and effective business decision-making.

A case study which aims to perform effort estimation of projects based on function point analysis, personnel metrics, programmer capability, language experience and cost metrics was conducted by Qi et al. [66]. The feature model that is built from the source code using the semantic of the text used can help in the clustering of the features into a hierarchy that can help a user identify the similarities and variations of features between different products. It has 3 steps, name harmonisation which extracts the semantics from the names of modules, identifying the features and lastly constructing the feature model with its constraints. Given an open source project completed by 5 contributors in 4 months, where 3 of 5 contributors are active contributors and remaining 2 people are inactive contributors. A proxy is used that 3 of 5 contributors provide 90% contributions, while the remaining devote 10% contributions.

Azzeh [67] proposed a hybrid model that combines classification and prediction stages using support vector machine and radial basis neural networks to estimate software project effort from Use Case Points (UCP). It emphasises the importance of predicting productivity based on environmental factors using historical data, rather than relying on fixed or guessed productivity ratios. The proposed model is validated empirically and demonstrated to significantly surpass previous UCP prediction models in terms of accuracy on all datasets. The model is trained and tested using a large number of observations from industrial and student projects.

8. Conclusion and future work

Acknowledging the gap between software clustering research and real-world application, we propose using REARRANGE to assist developers in making informed decisions about refactoring tasks. This research aims to bridge the gap in software modularisation research by providing developers with a concrete follow-up effort estimation to implement the suggested refactoring operations. With the estimated efforts, developers can make better-informed decisions to selectively choose refactoring operations within their limited time and budget constraints. By comparing MAE between REARRANGE (5.47) and Genetic effort estimation (453.31) as the next best alternative from Table 11, it is evident that REARRANGE performs better at estimating refactoring effort. The significant difference in MAE is due to Genetic effort estimation being a software effort estimation model, which can be argued as an unfair comparison. The lack of a direct comparison further highlights the absence and need for a refactoring effort-focused effort estimation model.

In this study, we aim to isolate and estimate refactoring efforts. The current proxy, defined as $\text{RefactoringTimeTaken} = \text{CommitTime} * \text{Percentage}$ (refactoring lines committed/total lines committed) works best when applied between two minor releases. Therefore, the scope of this study is confined to examining refactoring efforts between minor releases. When applied over the span of two major releases, this proxy may not perform as effectively due to the inclusion of various other activities such as requirement gathering, sprint activities, development, integration, and testing in the time span between two major releases. The reduced number of activities between minor releases contributes to the proxy's higher accuracy in those cases. It is plausible that refactoring efforts, when compared against major releases, could exhibit significantly different behaviours. Additional investigation into potential methodologies and proxies would allow the isolation of refactoring efforts over major releases to estimate refactoring effort.

We acknowledge that the accuracy of the proposed effort estimation might be influenced by a few factors, such as the experience and skills of the developers who will carry out the refactoring operations. As such, some margin of error is expected. Further exploration through in-depth use case analysis presents an opportunity to bolster the validation of REARRANGE, affirming its potential effectiveness. By undertaking additional validation efforts, we can enhance the justification for REARRANGE and furnish more compelling evidence of its value in software development practices.

The experiment results presented in this research show a feasible approach that software developers can implement to assist them in carrying out the suggested refactoring operations by software clustering algorithms. A potential use in practice is an extension in IDEs to suggest refactoring operations and the required effort, allowing developers to perform refactoring while implementing features. For model training, projects with significant refactoring are needed to predict the effort. In practice, REARRANGE only requires a single version of the source code as input, which can be easily applied. It is hoped that REARRANGE can serve as a framework for further analysis and improvements for research in software clustering-based modularisation.

CRedit authorship contribution statement

Alvin Jian Jia Tan: Writing – review & editing, Writing – original draft, Validation, Resources, Methodology, Investigation, Data curation, Conceptualization. **Chun Yong Chong:** Writing – review & editing, Validation, Supervision, Conceptualization. **Aldeida Aleti:** Writing – review & editing, Validation, Supervision, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Chun Yong Chong reports financial support was provided by Malaysia Ministry of Higher Education. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All of the data, source code, and model used in this research are provided as a replication package (<https://figshare.com/s/2ace70cefc45c28fe11>). To replicate the research, the notebooks provided can be used to obtain the complete dataset.

Acknowledgement

This work was carried out within the framework of the research project FRGS/1/2018/ICT01/MUSM/03/1 under the Fundamental Research Grant Scheme provided by the Ministry of Higher Education, Malaysia.

References

- [1] M. Hall, N. Walkinshaw, P. McMinn, Effectively incorporating expert knowledge in automated software remodularisation, *IEEE Trans. Softw. Eng.* 44 (7) (2018) 613–630.
- [2] N. Anquetil, T.C. Lethbridge, Comparative study of clustering algorithms and abstract representations for software remodularisation, *IEE Proc. - Softw.* 150 (3) (2003) 185–201.
- [3] K. Praditwong, M. Harman, Y. Xin, Software module clustering as a multi-objective search problem, *IEEE Trans. Softw. Eng.* 37 (2) (2011) 264–282.
- [4] J. Wu, A.E. Hassan, R.C. Holt, Comparison of clustering algorithms in the context of software evolution, in: 21st IEEE International Conference on Software Maintenance, ICSM'05, IEEE, 2005, pp. 525–535.
- [5] M. Fokaefs, N. Tsantalis, A. Chatzigeorgiou, J. Sander, Decomposing object-oriented class modules using an agglomerative clustering technique, in: IEEE International Conference on Software Maintenance, 2009, pp. 93–101.
- [6] R. Aull-Hyde, S. Erdogan, J.M. Duke, An experiment on the consistency of aggregated comparison matrices in ahp, *European J. Oper. Res.* 171 (1) (2006) 290–295.
- [7] P. Andritsos, V. Tzerpos, Information-theoretic software clustering, *IEEE Trans. Softw. Eng.* 31 (2) (2005) 150–165.
- [8] F. Meng, Y. Wang, C.Y. Chong, H. Yu, Z. Zhu, Evolution-aware constraint derivation approach for software remodularization, *ACM Trans. Softw. Eng. Methodol.* (2024) <http://dx.doi.org/10.1145/3676960>, just Accepted. [Online]. Available.
- [9] O. Maqbool, H. Babri, Hierarchical clustering for software architecture recovery, *IEEE Trans. Softw. Eng.* 33 (11) (2007).
- [10] C.Y. Chong, S.P. Lee, Automatic clustering constraints derivation from object-oriented software using weighted complex network with graph theory analysis, *J. Syst. Softw.* 133 (2017) 28–53.
- [11] M. Shtern, V. Tzerpos, Clustering methodologies for software engineering, *Adv. Softw. Eng.* 2012 (2012) 1.
- [12] S. Kaur, P. Singh, How does object-oriented code refactoring influence software quality? Research landscape and challenges, *J. Syst. Softw.* 157 (2019) 110394.
- [13] M. Hall, M.A. Khojaye, N. Walkinshaw, P. McMinn, Establishing the source code disruption caused by automated remodularisation tools, in: 2014 IEEE International Conference on Software Maintenance and Evolution, IEEE, 2014, pp. 466–470.
- [14] I. Candela, G. Bavota, B. Russo, R. Oliveto, Using cohesion and coupling for software remodularization: Is it enough? *ACM Trans. Softw. Eng. Methodol.* (TOSEM) 25 (3) (2016) 1–28.
- [15] M. Paixao, M. Harman, Y. Zhang, Y. Yu, An empirical study of cohesion and coupling: Balancing optimization and disruption, *IEEE Trans. Evol. Comput.* 22 (3) (2017) 394–414.
- [16] E. Tempero, T. Gorschek, L. Angelis, Barriers to refactoring, *Commun. ACM* 60 (10) (2017) 54–61.
- [17] C. Vassallo, F. Palomba, H.C. Gall, Continuous refactoring in ci: A preliminary study on the perceived advantages and barriers, in: 2018 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2018, pp. 564–568.
- [18] F. Sarro, A. Petrozziello, Linear programming as a baseline for software effort estimation, *ACM Trans. Softw. Eng. Methodol.* (TOSEM) 27 (3) (2018) 1–28.
- [19] P.A. Whigham, C.A. Owen, S.G. Macdonell, A baseline model for software effort estimation, *ACM Trans. Softw. Eng. Methodol.* (TOSEM) 24 (3) (2015) 1–11.
- [20] A.J.J. Tan, C.Y. Chong, A. Aleti, Closing the loop for software remodularisation-rearrange: An effort estimation approach for software clustering-based remodularisation, in: 2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings, ICSE-Companion, IEEE, 2023, pp. 326–327.
- [21] N. Teymourian, H. Izadkhah, A. Isazadeh, A fast clustering algorithm for modularization of large-scale software systems, *IEEE Trans. Softw. Eng.* (2020).
- [22] M. Shtern, V. Tzerpos, On the comparability of software clustering algorithms, in: Program Comprehension (ICPC), 2010 IEEE 18th International Conference on, IEEE, 2010, pp. 64–67.
- [23] C.Y. Chong, S.P. Lee, T.C. Ling, Efficient software clustering technique using an adaptive and preventive dendrogram cutting approach, *Inf. Softw. Technol.* 55 (11) (2013) 1994–2012.
- [24] F. Meng, Y. Wang, C.Y. Chong, H. Yu, Z. Zhu, CLUE: Customizing clustering techniques using machine learning for software modularization, in: Proceedings of the 15th Asia-Pacific Symposium on Internetware, Ser. Internetware '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 189–198, <http://dx.doi.org/10.1145/3671016.3674816>, [Online]. Available.
- [25] M. Shtern, V. Tzerpos, Factbase and decomposition generation, in: 2011 15th European Conference on Software Maintenance and Reengineering, 2011, pp. 111–120.
- [26] F. Beck, J. Melcher, D. Weiskopf, Identifying modularization patterns by visual comparison of multiple hierarchies, in: Program Comprehension (ICPC), 2016 IEEE 24th International Conference on, IEEE, 2016, pp. 1–10.
- [27] F. Beck, S. Diehl, On the impact of software evolution on software clustering, *Empir. Softw. Eng.* 18 (5) (2013) 970–1004.
- [28] R. Naseem, M.M. Deris, O. Maqbool, S. Shahzad, Euclidean space based hierarchical clusterers combinations an application to software clustering, *Cluster Comput.* 22 (3) (2019) 7287–7311.
- [29] A.J.J. Tan, C.Y. Chong, A. Aleti, E-SC4R: Explaining software clustering for remodularisation, *J. Syst. Softw.* (2021) 111162.
- [30] Z. Wen, V. Tzerpos, An optimal algorithm for mojo distance, in: Program Comprehension, 2003. 11th IEEE International Workshop on, IEEE, 2003, pp. 227–235.
- [31] F. Palomba, M. Tufano, G. Bavota, R. Oliveto, A. Marcus, D. Poshyvanyk, A. De Lucia, Extract package refactoring in aries, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, vol. 2, IEEE, 2015, pp. 669–672.
- [32] S.R. Chidamber, C.F. Kemerer, A metrics suite for object oriented design, *IEEE Trans. Softw. Eng.* 20 (6) (1994) 476–493.
- [33] M. Aniche, Java code metrics calculator (CK), 2015, Available in <https://github.com/mauricioaniche/ck>.
- [34] W. Jin, Y. Cai, R. Kazman, Q. Zheng, D. Cui, T. Liu, Enre a tool framework for extensible entity relation extraction, in: Proceedings of the 41st International Conference on Software Engineering Companion Proceedings, IEEE Press, 2019, pp. 67–70.
- [35] N. Tsantalis, M. Mansouri, L.M. Eshkevari, D. Mazinanian, D. Dig, Accurate and efficient refactoring detection in commit history, in: Proceedings of the 40th International Conference on Software Engineering, 2018, pp. 483–494.
- [36] N. Tsantalis, A. Ketkar, D. Dig, Refactoringminer 2.0, *IEEE Trans. Softw. Eng.* 48 (3) (2022) 930–950.
- [37] P. Alikhanifard, N. Tsantalis, A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools, 2024.
- [38] C. Schröder, A. van der Feltz, A. Panichella, M. Aniche, Search-based software re-modularization: a case study at adyen, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP, IEEE, 2021, pp. 81–90.
- [39] A. Martini, E. Sikander, N. Medlani, Estimating and quantifying the benefits of refactoring to improve a component modularity: a case study, in: 2016 42th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, 2016, pp. 92–99.
- [40] G. Sellitto, E. Iannone, Z. Codabux, V. Lenarduzzi, A. De Lucia, F. Palomba, F. Ferrucci, Toward understanding the impact of refactoring on program comprehension, in: 29th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2022.
- [41] E.A. AlOmar, A. Peruma, M.W. Mkaouer, C.D. Newman, A. Ouni, Behind the scenes: On the relationship between developer experience and refactoring, *J. Softw. Evol. Process* (2021) e2395.
- [42] E. LeDell, S. Poirier, H2O autoML: Scalable automatic machine learning, in: 7th ICML Workshop on Automated Machine Learning, AutoML, 2020.
- [43] M. Shepperd, S. MacDonell, Evaluating prediction systems in software project estimation, *Inf. Softw. Technol.* 54 (8) (2012) 820–827.
- [44] W.B. Langdon, J. Dolado, F. Sarro, M. Harman, Exact mean absolute error of baseline predictor, *marp0, Inf. Softw. Technol.* 73 (2016) 16–18.
- [45] S. Deeb, Estimating Refactoring Efforts for Architecture Technical Debt, West Virginia University, 2020.
- [46] A. Mittal, K. Parkash, H. Mittal, Software cost estimation using fuzzy logic, *ACM SIGSOFT Softw. Eng. Notes* 35 (1) (2010) 1–7.
- [47] J. Hayes, S. Patel, L. Zhao, A metrics-based software maintenance effort model, in: Eighth European Conference on Software Maintenance and Reengineering, 2004. CSMR 2004. Proceedings, 2004, pp. 254–258.
- [48] F.S. Alaa, A. Al-Afeef, A gp effort estimation model utilizing line of code and methodology for nasa software projects, in: 2010 10th International Conference on Intelligent Systems Design and Applications, IEEE, 2010, pp. 290–295.
- [49] R.F. Woolson, Wilcoxon Signed-Rank Test, in: Wiley Encyclopedia of Clinical Trials, 2007, pp. 1–3.
- [50] P. Royston, Approximating the shapiro-wilk w-test for non-normality, *Stat. Comput.* 2 (3) (1992) 117–119.
- [51] A. Arcuri, L. Briand, A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering, *Softw. Test. Verif. Reliab.* 24 (3) (2014) 219–250.
- [52] X. Wan, W. Wang, J. Liu, T. Tong, Estimating the sample mean and standard deviation from the sample size, median, range and/or interquartile range, *BMC Med. Res. Methodol.* 14 (2014) 1–13.
- [53] H. Zhang, L. Gong, S. Versteeg, Predicting bug-fixing time: an empirical study of commercial software projects, in: 2013 35th International Conference on Software Engineering, ICSE, IEEE, 2013, pp. 1042–1051.
- [54] M. Jørgensen, T. Halkjelsvik, K. Liestøl, When should we (not) use the mean magnitude of relative error (mmre) as an error measure in software development effort estimation? *Inf. Softw. Technol.* 143 (2022) 106784.
- [55] A.U. Jan, D.J. Dawood Jan, Y.H. Yousaf Hayat, A.S. Aisha Sadiqa, G.A. Ghaffar Ali, M.F. Muhammad Fayaz, Logarithmic transformation: a tool for normalizing residuals in anova models, *Sarhad J. Agric.* 30 (3) (2014) 375–378.
- [56] J.S. Molléri, K. Petersen, E. Mendes, An empirically evaluated checklist for surveys in software engineering, *Inf. Softw. Technol.* 119 (2020) 106240.

- [57] P. Runeson, M. Host, A. Rainer, B. Regnell, *Case Study Research in Software Engineering Guidelines and Examples*, John Wiley & Sons, 2012.
- [58] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *J. Syst. Softw.* 101 (2015) 193–220.
- [59] M. Aniche, E. Maziero, R. Durelli, V.H. Durelli, The effectiveness of supervised machine learning algorithms in predicting software refactoring, *IEEE Trans. Softw. Eng.* 48 (4) (2020) 1432–1450.
- [60] I. Kádár, P. Hegedus, R. Ferenc, T. Gyimóthy, A code refactoring dataset and its assessment regarding software maintainability, in: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, Vol. 1, SANER, IEEE, 2016, pp. 599–603.
- [61] P. Hegedús, I. Kádár, R. Ferenc, T. Gyimóthy, Empirical evaluation of software maintainability based on a manually validated refactoring dataset, *Inf. Softw. Technol.* 95 (2018) 313–327.
- [62] D.K. Moulla, A. Abran, et al., A data extraction algorithm from open source software project repositories for building duration estimation models: Case study of github, *Int. J. Softw. Eng. Appl. (IJSEA)* 11 (6) (2020).
- [63] V. Tawosi, A. Al-Subaih, F. Sarro, Investigating the effectiveness of clustering for story point estimation, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2022, pp. 827–838.
- [64] K.H. Kumar, K. Srinivas, An accurate analogy based software effort estimation using hybrid optimization and machine learning techniques, *Multimedia Tools Appl.* 82 (20) (2023) 30 463–30 490.
- [65] M.S. Khan, F. Jabeen, S. Ghouzali, Z. Rehman, S. Naz, W. Abdul, Metaheuristic algorithms in optimizing deep neural network model for software effort estimation, *IEEE Access* 9 (2021) 60 309–60 327.
- [66] F. Qi, X.-Y. Jing, X. Zhu, X. Xie, B. Xu, S. Ying, Software effort estimation based on open source projects: Case study of github, *Inf. Softw. Technol.* 92 (2017) 145–157.
- [67] M. Azzeh, A.B. Nassif, A hybrid model for estimating software project effort from use case points, *Appl. Soft Comput.* 49 (2016) 981–989.